CỘNG ĐỒNG HỌC TẬP TRÍ TUỆ NHÂN TẠO & TÀI CHÍNH ĐỊNH LƯỢNG CHO HỌC SINH PHỔ THÔNG VÀ ĐẠI HỌC



Ước Lượng Giá Nhà Ở Việt Nam
Từ Vị Trí Và Đặc Điểm Căn Nhà (Application)

Project Leader: *Nguyễn Hoàng Đăng Khoa*

AIE for Data: *Hoàng Đình Quốc Đoàn*

AIE for Model: *Trần Minh Triết*



AIQS Community

Ước Lượng Giá Nhà Ở Việt Nam Từ Vị Trí Và Đặc Điểm Căn Nhà

(Báo Cáo Ứng Dụng: Phân Tích Dữ Liệu Tin Rao Bất Động Sản Việt Nam 2024 & Mô Hình Gradient Boosting Cho Công Cụ Ước Lượng Giá Phổ Thông)

AIQS Community[†],
[†]Nguyễn Hoàng Đăng Khoa, Hoàng Đình Quốc Đoàn, Trần Minh Triết

Tóm tắt nội dung

Báo cáo giải quyết bài toán ước lượng tổng giá rao bán một căn nhà ở Việt Nam từ những thông tin mà người không chuyên vẫn khai báo được: tỉnh/thành, quận/huyện, diện tích, mặt tiền, đường vào, số tầng, số phòng, pháp lý và nội thất. Dữ liệu là bộ Vietnam Housing Dataset 2024 gồm 30 229 tin rao có giá từ 1 đến 11.5 tỷ VND, trong đó 74% tin nằm ở Hà Nội và TP.HCM. Bước kiểm tra dữ liệu phát hiện mặt tiền và đường vào đã bị điền khuyết trên toàn bộ dữ liệu trước khi chia tập, cùng 30 chuỗi tên tỉnh/thành chỉ khác bản chuẩn ở dấu chấm thừa hoặc khoảng trắng, làm một tỉnh bị tách thành nhiều nhóm; cả hai đã được sửa trước khi mô hình hóa (vẫn còn 18 tin ghi tỉnh/thành ở dạng viết tắt hoặc không phải tên tỉnh). Phân tích trên tập train cho thấy vị trí giải thích 62% phương sai của đơn giá/m^2, trong khi diện tích gần như không đi cùng tổng giá ($\rho = 0.19$): nhà càng rộng càng nằm ở vùng rẻ, hệ quả của việc dữ liệu chỉ giữ một khoảng giá. Lấy lại phường/xã, tên đường và tên dự án từ địa chỉ gốc là cải tiến lớn nhất (MAE giảm khoảng 6%). Khoảng 11% tin là tin đăng lại của cùng căn nhà; khi chia tập ngẫu nhiên, các bản đăng lại nằm ở cả train và test làm điểm số lạc quan (R^2 test 0.699, chỉ còn 0.677 khi bỏ tin trùng). Vì vậy nhóm chia lại dữ liệu theo nhóm tin cùng căn nhà và đánh giá lại với cấu hình không đổi. Mô hình cuối, trung bình XGBoost và LightGBM, đạt $R^2 = 0.676$ trên 3-fold cross-validation theo nhóm và $R^2 = 0.686$, MAE 0.921 tỷ VND trên tập test chia theo nhóm, **chưa đạt** mục tiêu $R^2 \geq 0.7$. Mô hình ước lượng trong sai số $\pm 20\%$ cho 70% căn nhà của tập test, so với 51% của quy tắc "đơn giá quận × diện tích" mà người mua và môi giới hay dùng; mô hình vẫn kéo giá về mức trung bình ở hai đầu phân phối. Khoảng dao động 80% hiệu chỉnh trên tập calibration (split conformal theo nhóm giá) phủ 79% giá thật trên test và rộng trung bình 2.85 tỷ VND. Kết quả phù hợp làm ước lượng tham khảo ban đầu, hiển thị cùng khoảng và cảnh báo trong ứng dụng, không thay thế thẩm định giá.

Mục lục

I Introduction **4**
- I.1 Bối cảnh và vấn đề . . . 4
- I.2 Mục tiêu và câu hỏi phân tích . . . 4
- I.3 Đóng góp chính . . . 4

II Domain Background **5**
- II.1 Bối cảnh lĩnh vực và thuật ngữ . . . 5
- II.2 Cơ chế vận hành của domain . . . 5
- II.3 Stakeholders và quyết định cần hỗ trợ . . . 6
- II.4 Kiến thức domain → Giả thuyết . . . 6
- II.5 Hiện trạng và các cách làm đang có . . . 6

III Data **7**
- III.1 Data Introduction . . . 7
- III.2 Data Dictionary . . . 8
- III.3 Data Quality Audit . . . 9

IV Exploratory Data Analysis **10**
- IV.1 Phân bố và thống kê mô tả . . . 10
- IV.2 Quan hệ giữa các biến . . . 11
- IV.3 Khác biệt theo vị trí và diện tích . . . 11
- IV.4 Kiểm chứng giả thuyết . . . 12

V Preprocessing & Feature Engineering **13**
- V.1 Tiền xử lý . . . 13
- V.2 Feature Engineering theo domain . . . 14
- V.3 Chia tập dữ liệu . . . 16

VI Insights: Rút Ra Được Gì Từ Data? **17**
- VI.1 Tổng hợp phát hiện chính . . . 17
- VI.2 Phát hiện bất ngờ và giả thuyết chưa được dữ liệu ủng hộ . . . 17
- VI.3 Nhóm, mẫu hình và phân khúc . . . 18
- VI.4 Từ insight đến quyết định . . . 18

VII Modeling **18**
- VII.1 Problem Formulation . . . 18
- VII.2 Baseline và lựa chọn mô hình . . . 19
- VII.3 Cải tiến theo góp ý: vị trí chi tiết và ensemble . . . 20
- VII.4 Mô hình học được điều gì? . . . 22

VIII Evaluation **23**
- VIII.1 Chỉ số kỹ thuật . . . 23
- VIII.2 Chỉ số theo domain và giá trị thực tế . . . 24
- VIII.3 Khoảng dự đoán . . . 24
- VIII.4 Error Analysis . . . 26

IX Application & Recommendations **27**
- IX.1 Cách sử dụng kết quả . . . 27
- IX.2 Kiểm thử với người dùng . . . 28
- IX.3 Khuyến nghị . . . 29
- IX.4 Giám sát và cập nhật . . . 29

X Limitations & Future Work **30**

A Data Dictionary đầy đủ **31**

B Reproducibility & Code Structure **31**

I Introduction

I.1 Bối cảnh và vấn đề

Nhà ở là khoản chi lớn nhất trong đời của phần lớn hộ gia đình Việt Nam, nhưng người mua lần đầu gần như không có công cụ nào để biết một mức giá rao là "hợp lý" hay không. Thông tin giá nằm rải rác trên các trang rao vặt, mỗi tin mô tả căn nhà theo một kiểu, và cùng một con số 6 tỷ VND có thể là một căn 40 m^2 trong ngõ ở Hà Nội hoặc một căn 120 m^2 ở ven Bình Dương. Người bán thì đặt giá rao dựa trên vài tin tương tự mà họ đã xem; môi giới định giá bằng kinh nghiệm về "đơn giá khu vực".

Cách làm phổ biến hiện nay là nhân đơn giá trung bình của khu vực với diện tích. Quy tắc này đơn giản nhưng bỏ qua số tầng, độ rộng đường vào, pháp lý, và đặc biệt sai lệch mạnh với nhà diện tích lớn ở vùng ven, nơi đơn giá thấp hơn hẳn trung bình quận. Một công cụ ước lượng dựa trên dữ liệu, giải thích được vì sao đưa ra con số đó và nói rõ khi nào không nên tin con số đó, sẽ giúp người dùng phổ thông có một điểm tham chiếu trước khi đi xem nhà hoặc thương lượng.

I.2 Mục tiêu và câu hỏi phân tích

Câu hỏi chính của báo cáo

- **Q1 (Domain):** Những yếu tố nào quyết định giá rao một căn nhà ở Việt Nam: vị trí, diện tích, kết cấu (số tầng, số phòng), hạ tầng tiếp cận (mặt tiền, đường vào), pháp lý hay yếu tố "mềm" như hướng nhà và nội thất?
- **Q2 (Data):** Bộ dữ liệu tin rao 2024 trả lời được đến đâu câu hỏi Q1? Dữ liệu bị thiếu, lệch mẫu và có nguy cơ rò rỉ ở đâu?
- **Q3 (Insight):** Data cho thấy yếu tố nào chi phối giá, yếu tố nào ít tác động hơn kỳ vọng, và mô hình sai nhiều với nhóm nhà nào?
- **Q4 (Action):** Người dùng ứng dụng, người bán và môi giới nên dùng con số ước lượng như thế nào, và ứng dụng cần cảnh báo gì?

I.3 Đóng góp chính

- **Kiểm tra và sửa dữ liệu có hệ thống:** phát hiện việc điền khuyết mặt tiền/đường vào bằng trung vị quận trên toàn bộ dữ liệu trước khi chia tập (nguy cơ rò rỉ và sai ngữ nghĩa), khôi phục giá trị gốc (`splits_v2`) và chuẩn hóa tên địa phương (106 → 76 chuỗi tên tỉnh/thành trên toàn bộ dữ liệu; vẫn còn 17 chuỗi viết tắt hoặc không phải tên tỉnh, gồm 18 tin); phát hiện 11% tin đăng trùng và 238 tin train có dấu hiệu bất thường về đơn giá, diện tích hoặc mức giá, trong đó có lỗi đơn vị gần như chắc chắn; chia lại dữ liệu theo nhóm tin cùng căn nhà (`splits_v3`) để đánh giá không bị lạc quan do tin đăng lại.
- **Vị trí chi tiết từ địa chỉ gốc:** tách lại phường/xã, tên đường, tên dự án đã bị bỏ khi rút gọn địa chỉ; đây là thay đổi cải thiện mô hình nhiều nhất, được kiểm chứng qua 3 seed và trên các tin không trùng.
- **Định lượng các yếu tố gắn với giá trong dữ liệu:** vị trí giải thích 62% phương sai đơn giá/m^2; làm rõ "nghịch lý diện tích" do dữ liệu bị giới hạn khoảng giá; chưa thấy pháp lý và hướng nhà làm khác tổng giá rõ rệt trong dữ liệu này (không có nghĩa là chúng không ảnh hưởng ngoài thực tế).

- **So sánh mô hình không rò rỉ:** từ baseline trung vị, hồi quy tuyến tính đến Random Forest, CatBoost, LightGBM, XGBoost, tất cả trên cùng các fold cố định, preprocessing học riêng trong từng fold; so sánh trực tiếp với quy tắc định giá của môi giới; đánh giá trên test sau khi khóa cấu hình, đánh giá lại với cách chia theo nhóm tin cùng căn, và xuất mô hình thành file.
- **Ứng dụng và khuyến nghị sử dụng:** ứng dụng ước tính giá bằng tiếng Việt, gồm bản Streamlit và bản web tĩnh chạy mô hình ngay trong trình duyệt (deploy trên Vercel, kết quả trùng với file mô hình); các cảnh báo (giá trị ngoài khoảng dữ liệu, phường ít dữ liệu, giá ngoài khoảng 1–11.5 tỷ VND) và kế hoạch giám sát.

II Domain Background

II.1 Bối cảnh lĩnh vực và thuật ngữ

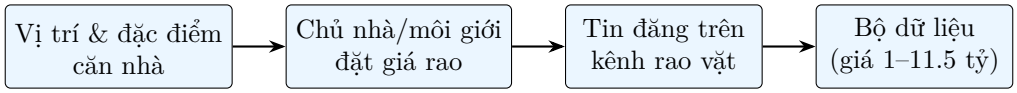
Thị trường nhà ở Việt Nam có hai đặc điểm chi phối cách đọc dữ liệu. Thứ nhất, giá gắn chặt với *đất*: ở nội đô Hà Nội và TP.HCM, phần lớn nhà là nhà phố (nhà ống) mặt tiền hẹp, xây nhiều tầng để bù diện tích đất nhỏ, nên giá phụ thuộc vào đơn giá đất của từng quận, thậm chí từng con đường. Thứ hai, tình trạng pháp lý quyết định khả năng sang tên và thế chấp: nhà có "sổ" được giao dịch, vay vốn dễ hơn nhà chỉ có hợp đồng mua bán. Bảng dưới giải thích các thuật ngữ xuất hiện trong dữ liệu.

Bảng 1: Bảng thuật ngữ domain bất động sản

Thuật ngữ	**Giải thích ngắn gọn**	**Xuất hiện ở biến**
Giá rao	Giá người bán/môi giới đăng trên tin; không phải giá giao dịch đã chốt.	`Price`
Đơn giá	Giá chia cho diện tích (triệu VND/m^2), thước đo môi giới dùng để so các khu vực.	`Price/Area`
Sổ đỏ/sổ hồng	Giấy chứng nhận quyền sử dụng đất, quyền sở hữu nhà ở; điều kiện để sang tên, thế chấp.	`Legal status` = `Have certificate`
Hợp đồng mua bán	Nhà mua theo hợp đồng với chủ đầu tư, thường là nhà dự án chưa cấp sổ.	`Legal status` = `Sale contract`
Mặt tiền	Chiều rộng cạnh nhà giáp đường (m); mặt tiền rộng thuận tiện kinh doanh.	`Frontage`
Đường vào / ngõ, hẻm	Độ rộng đường dẫn tới nhà (m); quyết định ô tô vào được hay không.	`Access Road`
Nhà phố (nhà ống)	Nhà mặt tiền hẹp, sâu, nhiều tầng; loại phổ biến ở đô thị Việt Nam.	`Floors`, `Area`
Hướng nhà	Hướng cửa chính/ban công; được quan tâm theo phong thủy.	`House/Balcony direction`
Tình trạng nội thất	Bán kèm nội thất đầy đủ (`Full`) hay cơ bản (`Basic`).	`Furniture state`

II.2 Cơ chế vận hành của domain

Một tin rao ra đời khi chủ nhà (thường qua môi giới) đặt giá dựa trên vị trí, kết cấu căn nhà và giá các tin tương tự xung quanh, rồi đăng lên kênh rao vặt. Bộ dữ liệu chỉ ghi lại *kết quả cuối* của quá trình này, và chỉ giữ các tin có giá trong một khoảng nhất định. Vì vậy dữ liệu phản ánh kỳ vọng của người bán, không phải giá thị trường đã khớp, và chịu ảnh hưởng của cách nguồn dữ liệu lọc tin.



Hình 1: Cơ chế hình thành giá rao và nơi dữ liệu được ghi lại

Trong chuỗi này, **vị trí** và **diện tích đất** là nguyên nhân chính của giá (đơn giá đất × diện tích); **số tầng, số phòng** phản ánh diện tích xây dựng và giá trị công trình; **pháp lý** ảnh hưởng tính thanh khoản; còn **hướng nhà, nội thất** là yếu tố phụ. Các biến bị bỏ trống trên tin rao không ngẫu nhiên: người đăng thường bỏ qua những trường không có lợi cho căn nhà.

II.3 Stakeholders và quyết định cần hỗ trợ

Bảng 2: Ai dùng kết quả và dùng để quyết định điều gì

Đối tượng	**Quyết định họ phải đưa ra**	**Họ cần gì từ data/mô hình**
Người mua nhà (phổ thông)	Có nên đi xem căn này không; trả giá ở mức nào.	Một khoảng giá tham khảo theo khu vực và đặc điểm căn; cảnh báo khi giá rao lệch xa.
Chủ nhà / người bán	Đặt giá rao ban đầu để bán được trong thời gian hợp lý.	Giá các căn tương tự đang rao; yếu tố nào thực sự làm tăng giá.
Môi giới	Tư vấn nhanh cho khách khi chưa có thẩm định.	Ước lượng nhanh hơn và ổn định hơn quy tắc đơn giá × diện tích.
Nhóm phát triển ứng dụng	Hiển thị con số nào, cảnh báo gì, khi nào không trả kết quả.	Sai số theo nhóm nhà, khoảng giá mô hình có cơ sở, các trường đầu vào quan trọng.

II.4 Kiến thức domain → Giả thuyết

Bảng 3: Danh sách giả thuyết

ID	**Giả thuyết**	**Cơ sở từ domain**	**Biến liên quan**
H1	Vị trí (tỉnh, quận) là yếu tố quyết định đơn giá/m^2.	Giá đất khác nhau nhiều lần giữa nội đô và vùng ven; môi giới định giá theo "đơn giá khu vực".	`City`, `District`
H2	Nhà diện tích lớn hơn có tổng giá cao hơn.	Giá ≈ đơn giá × diện tích.	`Area`
H3	Nhà có sổ có giá cao hơn nhà chỉ có hợp đồng mua bán.	Sổ giúp sang tên, thế chấp dễ; nhà chưa sổ chịu rủi ro pháp lý.	`Legal status`
H4	Nhà nhiều tầng, nhiều phòng tắm/WC có giá cao hơn.	Nhà phố xây cao để tăng diện tích sàn; nhiều WC phản ánh diện tích sàn và chất lượng.	`Floors`, `Bathrooms`, `Bedrooms`
H5	Hướng nhà "đẹp" theo phong thủy làm tăng giá.	Người mua Việt Nam quan tâm hướng Đông Nam, Nam.	`House direction`
H6	Nhà bán kèm nội thất đầy đủ có giá cao hơn.	Nội thất cộng thêm giá trị sử dụng ngay.	`Furniture state`

II.5 Hiện trạng và các cách làm đang có

- **Định giá theo phương pháp so sánh** (thẩm định viên, môi giới): so với vài giao dịch/tin rao tương tự gần đó rồi điều chỉnh theo khác biệt. Chính xác khi người làm hiểu khu vực, nhưng tốn thời gian, phụ thuộc kinh nghiệm cá nhân và không có sẵn cho người dùng phổ thông.

- **Bảng giá đất do UBND cấp tỉnh ban hành:** phục vụ tính thuế, phí, bồi thường; không được thiết kế để phản ánh giá rao của từng căn nhà cụ thể.
- **Quy tắc "đơn giá khu vực × diện tích":** cách ước lượng nhanh phổ biến nhất. Báo cáo dựng lại quy tắc này trên chính dữ liệu (trung vị đơn giá của quận học từ phần train của mỗi fold) để làm mốc so sánh: chỉ 52% căn nhà được ước lượng trong sai số ±20% (mục VIII.2).

Khoảng trống mà báo cáo lấp vào: một ước lượng dùng đồng thời nhiều đặc điểm căn nhà, được đánh giá trung thực trên dữ liệu chưa thấy, và đi kèm thông tin về độ tin cậy theo từng nhóm nhà.

Lỗi thường gặp ở mục Domain

Không gọi output là "giá thị trường" hay "giá giao dịch". Dữ liệu là giá rao do người bán đặt; nhóm chưa xác minh được dữ liệu đã được nguồn xử lý thế nào, nên mọi kết luận ở đây là về giá rao trong khoảng 1–11.5 tỷ VND.

III Data

III.1 Data Introduction

Dữ liệu là bộ *Vietnam Housing Dataset 2024* công bố trên Kaggle, gồm 30 229 tin rao bán nhà, mỗi dòng là một căn nhà với 11 thuộc tính mô tả và giá. Nhóm chọn bộ dữ liệu này vì có quy mô lớn, phủ nhiều tỉnh thành, và các thuộc tính đều là thông tin mà người dùng phổ thông tự khai báo được, đúng với kịch bản ứng dụng. Cột `Address` được tách thành `City` (phần cuối) và `District` (phần áp chót). Bản đầu tiên bỏ phần còn lại của địa chỉ; ở bước cải tiến, phường/xã, tên đường và tên dự án được tách lại từ `Address` gốc vì mang thông tin vị trí chi tiết hơn quận (mục V.2).

Bảng 4: Tổng quan bộ dữ liệu

Thuộc tính	Mô tả
Nguồn	Kaggle — Vietnam Housing Dataset 2024 (`data/vietnam_housing_dataset.csv`)
Khoảng thời gian	Tin rao năm 2024 (ảnh chụp một thời điểm, không có ngày đăng từng tin)
Đơn vị quan sát	Một dòng = một tin rao bán một căn nhà
Số dòng / số cột	30 229 dòng; 12 cột gốc → 13 cột sau khi tách địa chỉ
Phạm vi địa lý	106 chuỗi tên tỉnh/thành gốc (76 sau chuẩn hóa; đếm thủ công còn 59 tỉnh/thành), 341 chuỗi tên quận/huyện gốc; 74% tin ở Hà Nội và TP.HCM
Tần suất cập nhật	Không cập nhật (bộ dữ liệu tĩnh)
Biến mục tiêu	`Price`: tổng giá rao, đơn vị tỷ VND, khoảng 1.0–11.5
Giấy phép sử dụng	Theo điều khoản trang Kaggle của bộ dữ liệu; dùng cho mục đích học tập

III.2 Data Dictionary

Bảng 5: Các biến then chốt (tỷ lệ thiếu trên dữ liệu gốc)

Biến	**Kiểu**	**Đơn vị**	**Ý nghĩa theo domain**	**% thiếu**
`Price`	Số thực	tỷ VND	Tổng giá rao (biến mục tiêu)	0.0
`City`	Phân loại	–	Tỉnh/thành, tách từ địa chỉ	0.0
`District`	Phân loại	–	Quận/huyện, tách từ địa chỉ	0.0
`Area`	Số thực	m^2	Diện tích ghi trên tin (nhóm chưa xác minh là diện tích đất hay sàn)	0.0
`Floors`	Số nguyên	tầng	Số tầng; đại diện cho diện tích xây dựng	11.9
`Bathrooms`	Số nguyên	phòng	Số phòng tắm/WC	23.4
`Access Road`	Số thực	m	Độ rộng đường/ngõ vào nhà	44.0
`Legal status`	Phân loại	–	`Have certificate` / `Sale contract`	14.9
`Furniture state`	Phân loại	–	`Full` / `Basic`	46.7

Bảng đầy đủ 13 biến nằm ở Phụ lục A. Có tới 91% tin trong tập train bị thiếu ít nhất một trong 9 trường tùy chọn; chỉ 8.7% tin khai báo đầy đủ.

III.3 Data Quality Audit

Bảng 6: Kiểm tra chất lượng dữ liệu và phương án xử lý

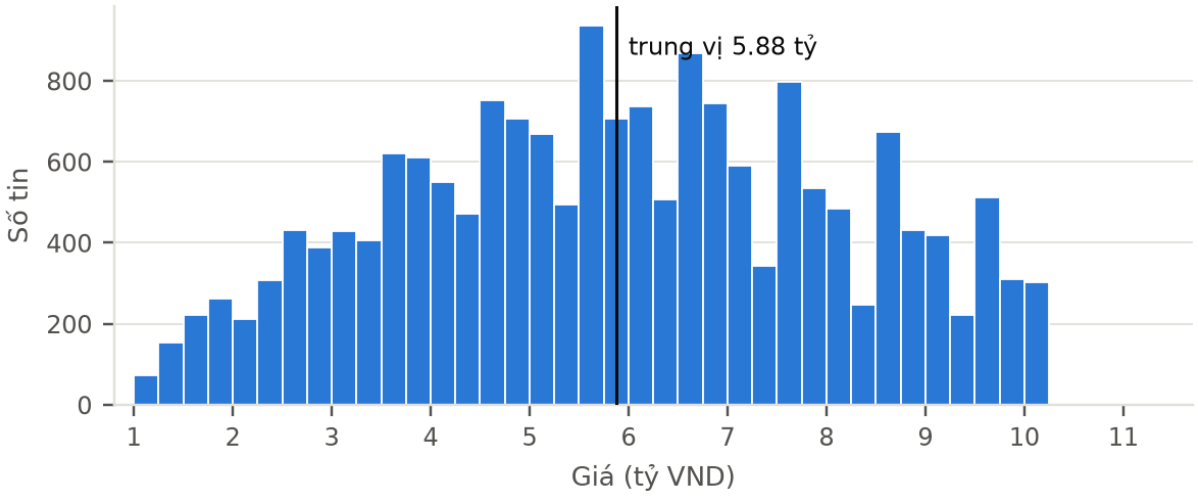
Vấn đề	Phát hiện	Mức độ	Cách xử lý	Lý do
Giá trị thiếu	9/12 cột thiếu 12–83%; hướng ban công thiếu 82.6%.	Cao	Điền trong pipeline, học riêng trong từng fold (số: mean sau chuẩn hóa; mặt tiền/đường vào: median hoặc để mô hình tự xử lý; phân loại: mode).	Không bỏ dòng; tránh dùng thông tin tập đánh giá.
Điền khuyết trước khi chia tập	Bản `_v1` đã điền `Frontage`/`Access Road` bằng trung vị quận trên toàn bộ 30 229 dòng; ô còn thiếu gán 0 và 2.0; 3 giá trị quan sát được ở dòng thiếu quận cũng bị ghi đè. Đã tái lập đúng công thức.	Cao	Khôi phục giá trị gốc thành `splits_v2`, giữ nguyên thành viên các tập; điền lại trong pipeline, học trong từng fold.	Chống rò rỉ; mặt tiền = 0 sai ngữ nghĩa.
Tên địa phương không thống nhất	"Hà Nội." và "Hà Nội" là hai giá trị; 509 dòng train đổi tên.	Trung bình	Chuẩn hóa NFC, khoảng trắng, dấu chấm cuối: 106 → 76 chuỗi tên tỉnh/thành; còn 17 chuỗi viết tắt hoặc rác (18 tin), chưa gộp.	Gộp nhóm bị tách giả tạo.
Tin đăng trùng	0 dòng trùng toàn bộ ở dữ liệu gốc, nhưng 11.1% dòng train trùng địa chỉ, diện tích và giá với dòng khác (nhiều khả năng một căn đăng nhiều lần); 10.5% dòng validation, 10.7% dòng test có bản trùng trong train.	Cao	Chia lại tập theo nhóm tin cùng căn (`splits_v3`) và đánh giá lại.	Bản trùng dễ đoán nên điểm khi chia ngẫu nhiên bị lạc quan; chưa xác minh được là cùng căn.
Giá trị bất thường	238 dòng train (1.3%) có ít nhất một dấu hiệu: đơn giá $< 1/3$ hoặc > 3 lần trung vị của quận có ≥ 30 tin train (151), diện tích < 15 hoặc > 400 m^2 (90), giá ở mức sàn/trần (7). Có lỗi đơn vị gần như chắc chắn (nhà 3 tầng ghi 4 m^2).	Trung bình	Đánh dấu, không xóa; cần nhóm dữ liệu xác minh nguồn.	Không xóa theo sai số mô hình; bỏ qua các dòng này R^2 OOF chỉ tăng 0.642 → 0.652.
Lệch mẫu	Giá chỉ trong 1–11.5 tỷ VND; 74% tin ở HN và TP.HCM; 226/302 nhóm tỉnh–quận có < 30 tin train.	Cao	Ghi rõ phạm vi áp dụng; cảnh báo trong app.	Mô hình không có cơ sở ngoài phạm vi này.
Nguy cơ rò rỉ	`row_id`, đơn giá/m^2 tính từ `Price`.	Cao	Loại khỏi đặc trưng; calibration chỉ dùng cho khoảng dự đoán; test chỉ dùng một lần, sau khi khóa mô hình.	Chỉ dùng thông tin có lúc dự đoán.

Ghi chú cho bảng: 17 chuỗi tỉnh/thành chưa gộp gồm chữ viết tắt ("TPHCM", "HN") và chuỗi không phải tên tỉnh ("giá 6ty", "Quận 8"). Một dòng có thể có nhiều dấu hiệu bất thường; nếu so đơn giá ở mọi quận, kể cả quận ít tin, số dòng lệch đơn giá là 194 và tổng là 273 (1.5%). Lỗi đơn vị trong bảng là một căn 3 tầng ghi 4 m^2, giá 6.4 tỷ VND. Số nhóm tỉnh–quận không tính 2 dòng thiếu tên tỉnh hoặc quận.

IV Exploratory Data Analysis

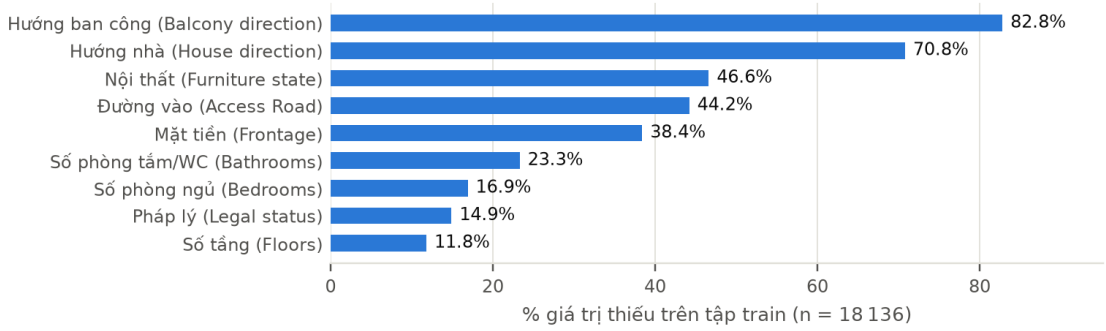
Toàn bộ phân tích trong mục này chỉ dùng tập train (18 136 dòng, `splits_v2`); tập calibration và test không được mở để tránh việc chọn mô hình dựa trên dữ liệu đánh giá cuối.

IV.1 Phân bố và thống kê mô tả



Hình 2: Phân bố giá rao trên tập train (bước 0.25 tỷ VND)

Giá có trung vị 5.88 tỷ VND, trung bình 5.87 tỷ VND, độ lệch chuẩn 2.21 tỷ VND và độ lệch (skewness) gần 0 (-0.02). Đây là dấu hiệu bất thường: giá bất động sản thực tế gần như luôn lệch phải với đuôi dài ở phân khúc cao cấp. Phân bố cân đối và bị chặn gọn trong 1–11.5 tỷ VND cho thấy nguồn dữ liệu đã lọc tin theo khoảng giá. Diện tích có trung vị 56 m^2 (từ 4 đến 595 m^2), còn đơn giá có trung vị 103 triệu VND/m^2 với 90% giá trị trong khoảng 28–206 triệu VND/m^2.



Hình 3: Tỷ lệ thiếu theo trường trên tập train

Hai trường hướng nhà, hướng ban công thiếu trên 70%; nội thất, đường vào, mặt tiền thiếu 38–47%. Các trường "mềm" bị bỏ trống nhiều nhất, khớp với cơ chế người đăng tin chỉ điền những gì họ cho là quan trọng.

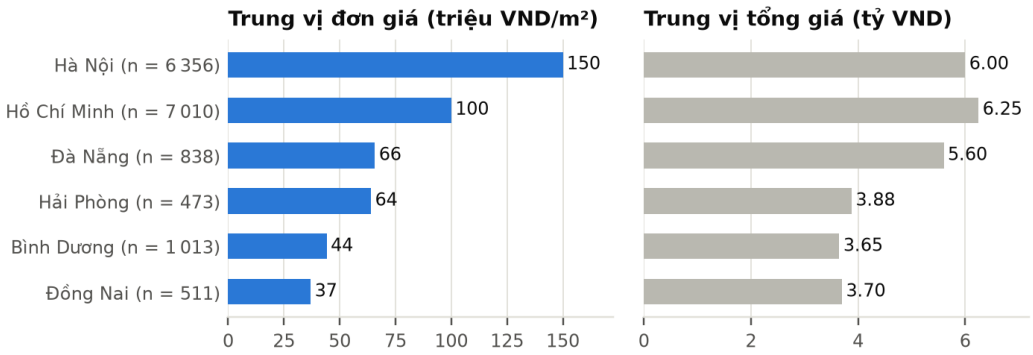
IV.2 Quan hệ giữa các biến

Bảng 7: Tương quan hạng Spearman giữa biến số và giá (tập train, bỏ dòng thiếu theo cặp)

Biến	n	ρ với giá	Nhận xét
Số phòng tắm/WC (`Bathrooms`)	13 908	0.462	Mạnh nhất trong nhóm biến số
Số phòng ngủ (`Bedrooms`)	15 077	0.425	Đi cùng diện tích sàn
Số tầng (`Floors`)	15 991	0.325	Nhà phố nhiều tầng giá cao hơn
Đường vào (`Access Road`)	10 126	0.245	Ngõ rộng, ô tô vào được
Diện tích (`Area`)	18 136	0.193	Yếu hơn kỳ vọng (xem IV.3)
Mặt tiền (`Frontage`)	11 177	0.112	Yếu
Diện tích với *đơn giá*	18 136	−0.697	Nhà càng rộng, đơn giá càng thấp

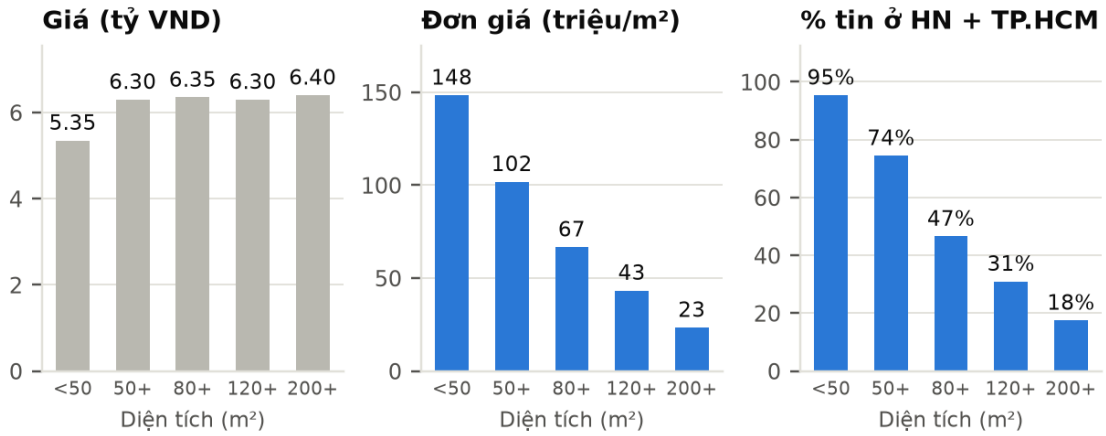
Với biến phân loại, nhóm dùng tỷ số tương quan η^2 (tỷ lệ phương sai của giá được giải thích bởi trung bình từng nhóm). Cặp tỉnh + quận giải thích 24.9% phương sai tổng giá và 61.8% phương sai đơn giá; tình trạng nội thất 3.7%; hướng nhà 0.5%; pháp lý gần 0%.

IV.3 Khác biệt theo vị trí và diện tích



Hình 4: Đơn giá và tổng giá theo 6 tỉnh/thành nhiều tin nhất (trung vị, tập train)

Đơn giá ở Hà Nội (150 triệu VND/m^2) gấp 4 lần Đồng Nai (37 triệu VND/m^2), nhưng tổng giá chỉ chênh 1.6 lần: người mua ở Hà Nội nhận diện tích nhỏ hơn hẳn (trung vị 40 m^2 so với 100 m^2). Nói cách khác, vị trí gắn chặt với *đơn giá*, còn diện tích "co giãn" để tổng giá vừa với khả năng chi trả.



Hình 5: Theo nhóm diện tích (m^2; "50+" nghĩa là 50 đến dưới 80, ...): tổng giá trung vị gần như đi ngang, đơn giá giảm mạnh và tỷ lệ tin ở Hà Nội + TP.HCM giảm theo

Từ 50 m^2 trở lên, tổng giá trung vị đứng yên quanh 6.3–6.4 tỷ VND dù diện tích tăng gấp bốn. Cùng lúc, tỷ lệ tin ở Hà Nội và TP.HCM giảm từ 95% (nhóm dưới 50 m^2) xuống 18% (nhóm từ 200 m^2). Diện tích lớn không làm giá cao hơn vì nhà diện tích lớn *chỉ lọt vào khoảng 1–11.5 tỷ VND khi nằm ở vùng giá rẻ.*

IV.4 Kiểm chứng giả thuyết

Bảng 8: Kết quả kiểm chứng giả thuyết (tập train)

ID	Phương pháp	Kết quả	Effect size	Kết luận
H1	Kruskal–Wallis, η^2 trên đơn giá	302 nhóm tỉnh–quận (bỏ 2 dòng thiếu tên), $p < 10^{-300}$	$\eta^2 = 0.618$ (lớn)	**Được ủng hộ**: vị trí gắn chặt với đơn giá trong dữ liệu này.
H2	Spearman `Area`–`Price`	$\rho = 0.193$; giá đi ngang từ 50 m^2	Nhỏ	**Không được ủng hộ ở mức toàn bộ dữ liệu**; phù hợp khi cố định vị trí (mục VII.4).
H3	Mann–Whitney, có sổ vs HĐMB	Trung vị 5.85 vs 6.00 tỷ VND; $p = 0.88$	Cliff's $\delta = 0.004$	**Chưa thấy khác biệt rõ** về tổng giá trong dữ liệu này.
H4	Spearman	`Bathrooms` 0.46; `Floors` 0.33	Trung bình	**Được ủng hộ** ở mức trung bình.
H5	Kruskal–Wallis, 8 hướng	$p = 0.0009$; trung vị 5.05–5.60 tỷ VND	$\eta^2 = 0.005$	**Có ý nghĩa thống kê nhưng hiệu ứng rất nhỏ** trong dữ liệu này.
H6	Mann–Whitney, Full vs Basic	Trung vị 6.20 vs 5.00 tỷ VND; $p < 10^{-78}$	Cliff's $\delta = 0.233$	**Được ủng hộ yếu**; mô hình gần như không dùng thêm khi đã có các biến khác (VII.4).

Tương quan không phải nhân quả

Nhà có sổ có *đơn giá* cao gấp đôi nhà hợp đồng mua bán (106.6 so với 52.9 triệu VND/m^2, Cliff's $\delta = 0.54$), nhưng không thể kết luận "có sổ làm giá tăng gấp đôi". Tin hợp đồng mua bán tập trung ở dự án vùng ven (Đồng Nai, Hưng Yên, Bình Dương chiếm 34%) với diện tích trung vị 100 m^2: vị trí và loại hình nhà là biến thứ ba giải thích phần lớn khác biệt. Tương tự, nội thất đầy đủ đi cùng nhà nhỏ nội đô, nên hiệu ứng của nội thất lẫn với hiệu ứng vị trí.
Các kết luận trong bảng chỉ nói về bộ tin rao năm 2024 này. "Chưa thấy khác biệt" không có nghĩa là yếu tố đó không ảnh hưởng giá ngoài thực tế: nhiều trường bị bỏ trống (hướng nhà thiếu 70%), giá trị được ghi thô (nội thất chỉ có "đầy đủ" hoặc "cơ bản"), không có biến tách nhà dự án và nhà thổ cư, và giá chỉ nằm trong 1–11.5 tỷ VND.

V Preprocessing & Feature Engineering

V.1 Tiền xử lý

Mọi bước tiền xử lý nằm trong một `Pipeline` của scikit-learn và được fit lại trong từng fold, nên không có thống kê nào (trung bình, mode, danh mục) được học từ phần dữ liệu dùng để chấm điểm.

1. **Chuẩn hóa địa danh:** Unicode NFC, gộp khoảng trắng, bỏ dấu chấm cuối cho `City`, `District`. Trên Random Forest, MAE validation của 120 dòng đổi tên giảm từ 1.030 xuống 0.853 tỷ VND.

2. **Biến số:** chuẩn hóa (StandardScaler) rồi điền thiếu bằng trung bình. Ở giai đoạn khám phá (`splits_v1`, Random Forest, validation): mean tốt hơn median (MAE 1.0188 so với 1.0212), tốt hơn KNN với $k = 3, 5, 15$ (1.0237–1.0254) và thêm cờ thiếu không cải thiện (1.0236).

3. **Mặt tiền, đường vào (thiếu 38–44%):** trên `splits_v2`, ba cách được so sánh bằng CV, tất cả học trong từng fold: trung vị, trung vị kèm cờ thiếu, và để mô hình tự xử lý giá trị thiếu. XGBoost dùng trung vị; LightGBM để tự xử lý (MAE giảm 0.47%, giữ qua 3 seed). Không gán mặt tiền = 0 vì thiếu không có nghĩa là nhà trong hẻm.

4. **Biến phân loại:** điền mode rồi One-Hot (`handle_unknown="ignore"` để quận mới không làm lỗi pipeline). Gán nhãn riêng `__MISSING__` cho ô thiếu cho kết quả kém hơn mode (1.0219 so với 1.0175). Sau mã hóa có 388 cột.

5. **Vị trí chi tiết:** phường/xã, tên đường, tên dự án được mã hóa bằng target encoding (giá trung bình có làm mượt của nhóm). Bộ mã hóa chỉ học trên phần train của fold và dùng cross-fitting 5 fold bên trong, nên một dòng không bao giờ được mã hóa bằng chính giá của nó.

V.2 Feature Engineering theo domain

Bảng 9: Các đặc trưng được tạo thêm hoặc đã thử. Mức thay đổi MAE là 3-fold CV so với cấu hình chưa có đặc trưng đó; mỗi đặc trưng được thử riêng và chỉ nhận khi MAE giảm, thắng ít nhất 2/3 fold, giữ được qua 3 seed và R^2 không giảm.

Đặc trưng	Cách tính	Ý nghĩa theo domain	Trạng thái
`City`, `District`	Tách `Address` theo dấu phẩy, lấy 2 phần cuối	Đại diện đơn giá đất khu vực (H1)	Dùng
Địa danh chuẩn hóa	NFC, bỏ dấu chấm, khoảng trắng	Gộp các chuỗi chỉ khác dấu chấm, khoảng trắng (chưa gộp viết tắt như "TPHCM")	Dùng
Phường/xã	Phần đứng trước quận trong `Address`; target encoding theo tỉnh + quận + phường	Giá đất chênh nhiều giữa các phường trong cùng quận	**Dùng**: MAE −4.5% (XGBoost), −5.1% (LightGBM)
Tên dự án	Phần đầu "Dự án …" (12% tin); target encoding theo tỉnh + dự án	Nhà dự án có mặt bằng giá riêng	**Dùng**: MAE −0.8% / −0.9%
Tên đường	Phần đứng trước phường (86% tin); target encoding theo tỉnh + quận + đường	Mặt đường lớn, trục thương mại	**Dùng**: thêm −0.5% / −0.9% khi đã có phường
`City_District`	Ghép tỉnh và quận (One-Hot hoặc target encoding)	Phân biệt quận trùng tên giữa các tỉnh	Đã thử: trên `splits_v1` target encoding giúp LightGBM (−0.6%), One-Hot không giữ được qua 3 seed với XGBoost; trên `splits_v2` không giúp cả hai, bỏ
Diện tích × vị trí	`Area` × mức giá quận (target encoding)	Tác động của diện tích phụ thuộc vị trí	Đã thử, bỏ
Diện tích × số tầng	`Area` × `Floors`	Chỉ là tích số, không phải diện tích sử dụng thực tế	Đã thử, bỏ
Mật độ phòng	(`Bedrooms` + `Bathrooms`) / `Area`	Nhà xây kín, nhiều phòng	Đã thử, bỏ
log(`Area`)	Thay `Area`	Mô hình cây gần như không đổi khi biến đổi đơn điệu	Đã thử, bỏ (XGBoost giống hệt)
Cờ thiếu dữ liệu	`isna()` cho biến số	"Không khai báo" có thể mang thông tin	Đã thử, bỏ
Nhãn `__MISSING__`	Danh mục riêng cho ô trống	Như trên, cho biến phân loại	Đã thử, bỏ
Đơn giá/m^2	`Price / Area`	Thước đo của môi giới	Chỉ dùng cho EDA và rà soát (rò rỉ nhãn)

Pipeline tiền xử lý của thành phần XGBoost trong mô hình cuối

```
import re
import unicodedata
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
```

```
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (FunctionTransformer, OneHotEncoder,
                                   StandardScaler, TargetEncoder)

NUM = ["Area", "Floors", "Bedrooms", "Bathrooms"]
FA = ["Frontage", "Access Road"]          # thieu 38-44%
CAT = ["House direction", "Balcony direction", "Legal status",
       "Furniture state", "City", "District"]
# Khoa vi tri chi tiet, ghep tu City, District va phan tach tu Address
TE = ["City_District_Ward", "City_District_Street", "City_Project"]

def normalize_location_frame(X):
    # Chuan hoa ten dia phuong: NFC, gop khoang trang, bo dau cham cuoi
    X = X.copy()
    def norm(v):
        if pd.isna(v):
            return v
        v = unicodedata.normalize("NFC", str(v))
        return re.sub(r"\s+", " ", v).strip().rstrip(".").strip()
    for col in ["City", "District"]:
        X[col] = X[col].map(norm)
    return X

def build_location_keys(X):
    # Ward, Street, Project tach tu Address; thieu thi khoa cung thieu
    X = normalize_location_frame(X)
    loc = X["City"] + " | " + X["District"]
    X["City_District_Ward"] = loc + " | " + X["Ward"]
    X["City_District_Street"] = loc + " | " + X["Street"]
    X["City_Project"] = X["City"] + " | " + X["Project"]
    return X

# So: chuan hoa roi dien mean; mat tien, duong vao: dien median;
# phan loai: dien mode roi One-Hot; vi tri chi tiet: target encoding
num_pipe = Pipeline([("scaler", StandardScaler()),
                     ("imputer", SimpleImputer())])
fa_pipe = Pipeline([("scaler", StandardScaler()),
                    ("imputer", SimpleImputer(strategy="median"))])
cat_pipe = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                     ("onehot", OneHotEncoder(handle_unknown="ignore"))])
te_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="constant",
                              fill_value="__MISSING__")),
    ("encoder", TargetEncoder(target_type="continuous", smooth="auto",
                              cv=KFold(5, shuffle=True, random_state=42))),
])

preprocessing = Pipeline([
    # chuan hoa dia danh roi ghep cac khoa vi tri chi tiet
    ("features", FunctionTransformer(build_location_keys)),
    ("columns", ColumnTransformer([("num", num_pipe, NUM),
                                   ("fa", fa_pipe, FA),
                                   ("cat", cat_pipe, CAT),
                                   ("target", te_pipe, TE)])),
])
# Thanh phan LightGBM giong het, chi khac: khong dien FA (de mo hinh tu xu ly
    )
```

V.3 Chia tập dữ liệu

Mỗi dòng là một căn nhà riêng và mục tiêu là ước lượng cho căn nhà khác trong cùng bối cảnh, không dự báo tương lai, nên nhóm chia ngẫu nhiên với `seed = 42` và cố định từ đầu dự án:

Bảng 10: Giao thức chia dữ liệu

Tập	**Tỷ lệ (v3)**	**Số dòng v2 → v3**	**Vai trò**
Train	62.5%	18 136 → 18 880	Fit tiền xử lý và mô hình; 3-fold CV cố định để chọn mô hình, tham số
Validation	14.1%	4 535 → 4 268	So sánh nhanh ở giai đoạn khám phá; sau đó chỉ dùng để kiểm tra lại lựa chọn từ CV
Calibration	9.3%	3 023 → 2 807	Hiệu chỉnh khoảng dự đoán sau khi khóa mô hình (notebook 04, 05)
Test	14.1%	4 535 → 4 274	Đánh giá cấu hình đã khóa trên `splits_v2` (20/10/2026); đánh giá lại trên `splits_v3` (26/10/2026), cấu hình không đổi

Bốn tập không giao nhau theo `row_id` và được lưu thành file. Khi sửa dữ liệu, nhóm tạo phiên bản mới (`splits_v2`) nhưng giữ nguyên thành viên từng tập. Các bản ban đầu chia ngẫu nhiên theo dòng, nên một căn đăng nhiều lần có thể có tin ở cả train và test. Theo góp ý phản biện, notebook 05 tạo `splits_v3`: hai tin được coi là cùng căn khi cùng địa chỉ và diện tích, và cùng giá hoặc cùng số tầng, phòng ngủ, phòng tắm (địa chỉ không có số nhà nên không xác định được căn chắc chắn); các tin nối nhau qua điều kiện này thuộc một nhóm. Mỗi nhóm nằm trọn ở tập chứa nhiều tin nhất của nó trong `splits_v2`, nên chỉ 2 345 dòng (7.8%) đổi tập. Sau khi chia lại, không còn tin nào ở validation, calibration hay test trùng địa chỉ, diện tích và giá với train (trước đó khoảng 10.7%). Các fold CV và phần 15% dùng để dừng sớm cũng chia theo nhóm. Phần EDA và các bước chọn mô hình ở mục IV–VII được làm trên train của `splits_v2`; mọi con số đánh giá ở mục VIII và ứng dụng dùng `splits_v3`.

VI Insights: Rút Ra Được Gì Từ Data?

VI.1 Tổng hợp phát hiện chính

Bảng 11: Các phát hiện chính từ data

#	Phát hiện	Bằng chứng	Giải thích theo domain	Hành động	Độ tin cậy
1	Vị trí gắn chặt với đơn giá	$\eta^2 = 0.62$ trên đơn giá; HN 150 vs Đồng Nai 37 triệu VND/m^2	Giá đất theo khu vực là thành phần lớn nhất của giá nhà	Bắt buộc nhập tỉnh và quận trong app	Cao
2	"Nghịch lý diện tích"	Giá đi ngang 6.3 tỷ VND từ 50 m^2; ρ(diện tích, đơn giá) $= -0.70$	Dữ liệu chỉ giữ 1–11.5 tỷ VND: nhà rộng chỉ lọt vào nếu ở vùng rẻ	Không đọc diện tích tách khỏi vị trí	Cao
3	Kết cấu quan trọng hơn diện tích đơn lẻ	ρ WC 0.46, tầng 0.33 so với diện tích 0.19	Nhà phố xây cao bù đất nhỏ; số tầng/WC đo diện tích sàn	Hỏi số tầng, WC; cảnh báo khi bỏ trống	Trung bình
4	Chưa thấy pháp lý làm khác tổng giá trong dữ liệu này	$\delta = 0.004$, $p = 0.88$	"Hợp đồng mua bán" là nhà dự án vùng ven, diện tích lớn; dữ liệu không tách được nhà dự án và thổ cư	Chưa đủ cơ sở coi "có sổ" là điểm cộng giá riêng	Trung bình
5	Mô hình kéo giá về trung bình	Nhà < 3 tỷ bị đoán cao $+0.87$; nhà ≥ 9 tỷ bị đoán thấp -1.68	Thiếu thông tin chất lượng, mặt đường; dữ liệu bị chặn 1–11.5 tỷ VND	Hiện khoảng giá và cảnh báo ở hai đầu	Cao
6	Mô hình ít dùng hướng nhà, nội thất	Permutation importance < 0.01 tỷ VND	Lẫn với vị trí; nhiều tin bỏ trống; mô hình ít dùng chưa có nghĩa là không quan trọng ngoài thực tế	Để là trường tùy chọn	Trung bình
7	Vị trí trong quận quan trọng	Thêm phường/xã: MAE -4.5%, R^2 $+0.03$; thêm dự án và đường: thêm khoảng -1.5% (XGBoost)	Đơn giá đất khác nhau giữa các phường, các trục đường	Hỏi phường/xã, tên đường trong app	Cao
8	Khoảng 11% tin là tin đăng trùng	Trùng địa chỉ, diện tích, giá; với XGBoost cơ sở, dòng có bản trùng có MAE 0.69 so với 1.01	Một căn được đăng lại nhiều lần	Đã chia tập theo nhóm tin cùng căn và đánh giá lại: R^2 test 0.699 $\rightarrow$ 0.686	Trung bình

VI.2 Phát hiện bất ngờ và giả thuyết chưa được dữ liệu ủng hộ

Nghịch lý diện tích (H2). Kỳ vọng ban đầu là "nhà càng rộng càng đắt". Ở mức toàn bộ dữ liệu, điều này gần như không đúng: tương quan chỉ 0.19 và tổng giá đi ngang từ 50 m^2. Nguyên nhân là *selection bias* do nguồn chỉ giữ tin 1–11.5 tỷ VND: một căn 200 m^2 ở nội đô Hà Nội có giá vượt xa 11.5 tỷ VND nên không có mặt trong dữ liệu; căn 200 m^2 còn lại thường ở Bình Dương, Đồng Nai. Khi giữ vị trí cố định, diện tích lại là đặc trưng quan trọng nhất của mô hình (mục VII.4). Đây là trường hợp điển hình của nghịch lý Simpson: quan hệ trong từng nhóm khác với quan hệ khi gộp các nhóm.

Pháp lý (H3) và phong thủy (H5). Hai yếu tố được bàn nhiều khi mua nhà lại chưa thấy giải thích rõ tổng giá trong dữ liệu này. Với pháp lý, khác biệt đơn giá có thật nhưng bị nhiễu bởi vị trí và loại hình nhà; dữ liệu không có biến "nhà dự án / nhà thổ cư" để tách hai hiệu ứng. Với hướng nhà, khác biệt trung vị giữa các hướng (5.05–5.60 tỷ VND) nhỏ hơn nhiều so với độ biến động trong từng hướng, và 70% tin không ghi hướng. Vì vậy nhóm chỉ kết luận rằng dữ liệu này chưa cho thấy ảnh hưởng rõ, không kết luận rằng pháp lý hay hướng nhà

không ảnh hưởng giá ngoài thực tế.

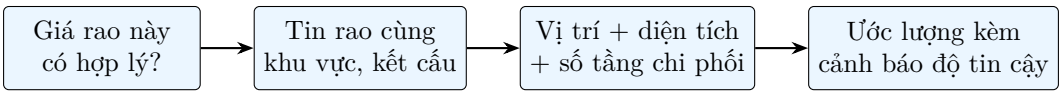
VI.3 Nhóm, mẫu hình và phân khúc

Bảng 12: Phân khúc theo vị trí và diện tích (train của `splits_v3`; sai số từ dự đoán out-of-fold theo nhóm của mô hình cuối)

Phân khúc	n	Giá (tỷ)	Đơn giá (tr/m^2)	MAE	MdAPE
Nhà nhỏ Hà Nội (<50 m^2)	4 966	5.68	158	0.83	11.5%
Nhà phố Hà Nội (50–100 m^2)	1 507	7.60	129	1.13	12.5%
Nhà nhỏ TP.HCM (<50 m^2)	2 104	4.65	130	0.77	12.9%
Nhà phố TP.HCM (50–100 m^2)	4 328	6.59	99	0.93	10.9%
Nhà lớn TP.HCM (≥ 100 m^2)	828	7.70	63	1.14	11.9%
Tỉnh khác (50–100 m^2)	2 668	4.30	60	0.90	14.2%
Tỉnh khác (≥ 100 m^2)	2 026	5.50	38	1.12	16.3%

Có thể gọi tên bốn nhóm theo ngôn ngữ thị trường: *nhà nhỏ Hà Nội* (đất nhỏ, đơn giá rất cao), *nhà phố TP.HCM* (nhóm đông nhất, mô hình dự đoán tốt nhất, MdAPE 10.9%), *nhà đất vùng ven* (diện tích lớn, đơn giá thấp, sai số tương đối cao hơn) và *nhà dự án chưa có sổ* (545 tin, đơn giá 53 triệu VND/m^2). MAE tuyệt đối tăng theo giá trị căn nhà, nhưng sai số tương đối lớn nhất lại nằm ở các tỉnh ngoài Hà Nội và TP.HCM, nơi mỗi quận có ít tin để mô hình học.

VI.4 Từ insight đến quyết định



Hình 6: Chuỗi giá trị của báo cáo: từ câu hỏi của người mua đến con số có kèm điều kiện sử dụng

Một insight tốt phải trả lời được ba câu

(1) **Bằng chứng:** tổng giá đi ngang quanh 6.3 tỷ VND từ 50 m^2 trong khi đơn giá giảm từ 148 xuống 23 triệu VND/m^2 và tỷ lệ tin ở Hà Nội + TP.HCM giảm từ 95% xuống 18%. (2) **Cơ sở domain:** giá nhà $\approx$ đơn giá đất khu vực $\times$ diện tích; dữ liệu lọc theo khoảng giá nên nhà rộng chỉ xuất hiện ở vùng rẻ. (3) **Hành động:** không nên so sánh giá hai căn chỉ theo diện tích; ứng dụng bắt buộc nhập vị trí và cảnh báo khi ước lượng nằm ngoài khoảng giá mà dữ liệu bao phủ.

VII Modeling

VII.1 Problem Formulation

Bài toán là hồi quy có giám sát trên dữ liệu bảng. Với mỗi căn nhà i có vector đặc trưng $\mathbf{x}_i$ gồm 12 biến người dùng khai báo (6 số, 6 phân loại) cùng phường/xã, tên đường, tên dự án tách từ địa chỉ, mô hình học hàm f sao cho

$$\hat{P}_i = f(\mathbf{x}_i), \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|P_i - \hat{P}_i\right|,$$

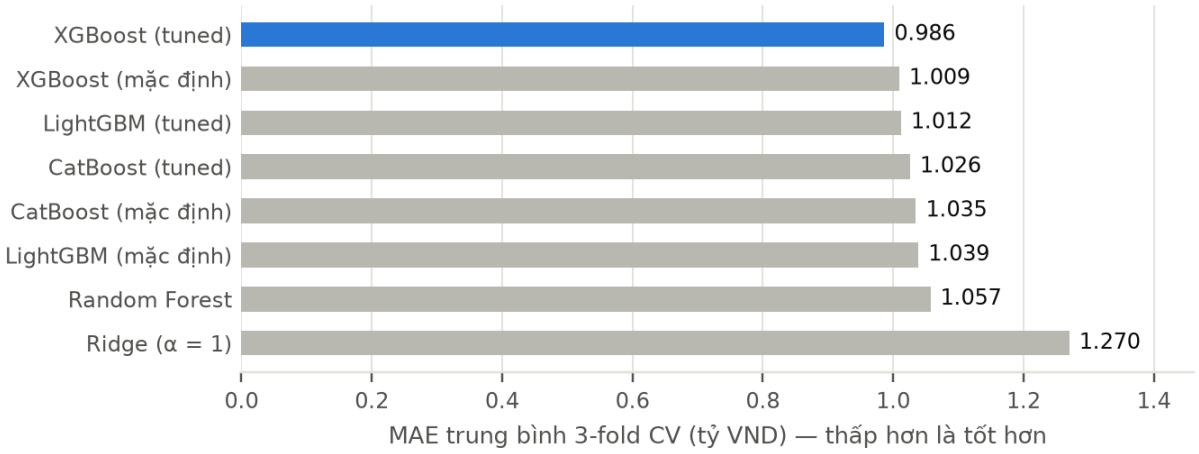
trong đó P_i là giá rao (tỷ VND). MAE là metric chọn mô hình vì cùng đơn vị với giá và người dùng hiểu ngay ("sai trung bình bao nhiêu tỷ"); RMSE theo dõi lỗi lớn; R^2 là tiêu chí nghiệm thu của dự án ($R^2 \geq 0.7$ trên tập test). Vì cấu hình tốt nhất theo MAE chưa chắc tốt nhất theo R^2, ở giai đoạn cải tiến (mục VII.3) một thay đổi chỉ được nhận khi MAE giảm *và* R^2 CV không giảm; grid search lượt 1 chọn theo MAE. Ràng buộc: chỉ dùng thông tin người dùng khai báo được lúc dự đoán; pipeline phải chịu được trường bỏ trống và quận chưa từng thấy.

VII.2 Baseline và lựa chọn mô hình

Nhóm đi từ đơn giản đến phức tạp. Giai đoạn đầu so sánh nhanh trên tập validation; sau đó chuyển sang 3-fold cross-validation trên tập train để có ước lượng ổn định hơn và dùng cho tinh chỉnh tham số.

Bảng 13: Giai đoạn khám phá: so sánh trên tập validation (`splits_v1`, 4 535 dòng). Ba mô hình boosting dùng chính tập validation để dừng sớm nên điểm của chúng hơi lạc quan.

Mô hình	**MAE**	**RMSE**	R^2	**Ghi chú**
Baseline: trung vị giá train (5.88 tỷ)	1.824	2.190	0.000	Mốc sàn
Hồi quy tuyến tính	1.263	1.643	0.437	Thiếu quan hệ phi tuyến
Ridge ($\alpha = 1$)	1.256	1.624	0.450	Điều chuẩn cải thiện rất ít
Random Forest (mặc định)	1.024	1.388	0.598	Train R^2 0.935: overfit
CatBoost (3 000 vòng)	0.991	1.315	0.639	Fit chậm nhất
LightGBM (early stopping)	0.986	1.320	0.637	Fit 3 giây
XGBoost (3 000 vòng)	0.981	1.327	0.633	MAE thấp nhất



Hình 7: So sánh 8 pipeline trên cùng 3 fold cố định của tập train (`splits_v1`)

Tinh chỉnh bằng grid search trên cùng 3 fold, chọn theo MAE trung bình; trong mỗi fold, số cây được chọn bằng early stopping trên 15% phần train của fold rồi refit, nên phần đánh giá không tham gia chọn số cây.

Bảng 14: Kết quả grid search lượt 1 (MAE 3-fold CV, tỷ VND)

Mô hình	Số cấu hình	Cấu hình chọn	Trước	Sau
Random Forest	144	`max_features`=0.7, cây không giới hạn độ sâu	1.057	1.057
CatBoost	27	`depth`=8, `l2_leaf_reg`=3, `random_strength`=2	1.035	1.026
LightGBM	36	`num_leaves`=63, `min_child_samples`=10	1.039	1.012
XGBoost	36	`max_depth`=9, `subsample`=0.8, `colsample_bytree`=0.7	1.009	**0.986**

Hai ứng viên được giữ lại là **XGBoost** (sai số thấp nhất) và **LightGBM** (sai số gần tương đương, huấn luyện nhanh hơn khoảng 8 lần). Sau khi sửa lỗi điền khuyết trước khi chia tập, cả hai được huấn luyện lại trên `splits_v2` và lưu checkpoint ngày 16/10/2026:

Bảng 15: Hai ứng viên trên `splits_v2` (trung bình 3 fold)

Mô hình	MAE	RMSE	R^2	Train MAE	Số cây chọn (3 fold)
XGBoost	0.984	1.319	0.645	0.553	738 / 1 141 / 804
LightGBM	1.016	1.352	0.627	0.757	384 / 343 / 367

Sau khi bỏ giá trị điền sẵn, XGBoost không giảm chất lượng (MAE 0.986 → 0.984), cho thấy điểm số trước đó không bị thổi phồng đáng kể bởi lỗi dữ liệu. Cấu hình XGBoost dưới đây được giữ nguyên cho mô hình cuối; mục VII.3 mô tả các thay đổi tiếp theo.

Cấu hình XGBoost được chọn

```python
from xgboost import XGBRegressor
from sklearn.pipeline import Pipeline

model = XGBRegressor(
    n_estimators=3000,      # tran tren; so cay that chon bang early stopping
    learning_rate=0.05,
    max_depth=9,
    min_child_weight=1,
    subsample=0.8,
    colsample_bytree=0.7,
    tree_method="hist",
    eval_metric="mae",
    random_state=42,
)
pipeline = Pipeline([("preprocessing", preprocessing), ("model", model)])

# Trong moi fold cua 3-fold CV:
#   1. tach 15% phan fit de chon so cay (dung sau 100 vong khong cai thien)
#   2. refit voi so cay da chon tren toan bo phan fit
#   3. cham MAE, RMSE, R2 tren phan assessment cua fold
```

VII.3 Cải tiến theo góp ý: vị trí chi tiết và ensemble

Sau góp ý của người hướng dẫn, nhóm tạm dừng grid search rộng, giữ XGBoost và LightGBM, và chỉ thử các thay đổi xuất phát từ một giả thuyết cụ thể. Mỗi thay đổi được so trên cùng 3 fold với cấu hình ngay trước nó và chỉ được nhận khi đủ bốn điều kiện: (1) MAE trung bình

giảm; (2) thắng ít nhất 2/3 fold; (3) giữ được lợi thế khi đổi seed mô hình (seed 42, 7, 2026), vì riêng việc chạy lại XGBoost trên máy khác số luồng CPU đã làm MAE mỗi fold lệch tới 0.004; (4) R^2 CV không giảm.

Bảng 16: Tiến trình trên `splits_v2` (3-fold CV, trung bình). Mức giảm MAE trong ngoặc là trung bình 3 seed.

Cấu hình	**MAE**	**RMSE**	R^2	**Thay đổi MAE**
XGBoost, mặt tiền/đường vào điền trung vị trong fold	0.983	1.325	0.642	–
+ phường/xã	0.938	1.270	0.671	−4.5% (−4.7%)
+ phường/xã + dự án	0.930	1.261	0.676	−5.4% (−5.3%)
+ phường/xã + dự án + đường	0.925	1.256	0.678	thêm −0.5% (−0.9%)
LightGBM tương tự (không điền mặt tiền/đường vào)	0.926	1.255	0.679	
Trung bình 0.5 XGBoost + 0.5 LightGBM	**0.915**	**1.243**	**0.685**	thêm −1.0%

- **Vị trí chi tiết là nguồn cải thiện lớn nhất**, đúng như dự đoán rằng việc rút gọn địa chỉ đã làm mất thông tin. Lợi ích của phường giữ nguyên trên các tin không có bản trùng trong phần train của fold (−4.5%). Lợi ích của tên đường với XGBoost chỉ còn −0.1% trên các tin không trùng, nghĩa là một phần đến từ việc "nhớ" giá của tin đăng lại cùng con đường; LightGBM giữ được −0.5%.

- **Bị loại:** `City_District` (One-Hot hoặc target encoding), diện tích × vị trí, diện tích × số tầng, mật độ phòng, log(`Area`), tinh chỉnh thêm tham số XGBoost (`colsample_bytree` = 0.5 tốt hơn ở seed 42 nhưng không giữ được qua 3 seed).

- **Chọn theo MAE hay theo R^2:** trên 7 lượt tìm kiếm, chọn theo RMSE/R^2 chỉ khác chọn theo MAE ở 2 lượt grid đầu, và R^2 mất đi khi chọn theo MAE dưới 0.002. Dừng sớm theo RMSE thay vì MAE gần như không thay đổi kết quả (R^2 thay đổi dưới 0.002, MAE không giảm) nên không được nhận.

- **Log giá:** huấn luyện trên log(`Price`), đổi dự đoán về giá gốc trước khi chấm. Ở một bước, cách này giảm MAE của XGBoost 0.37% nhưng luôn làm R^2 giảm 0.007–0.010, vì kéo dự đoán của nhà đắt xuống thấp hơn. Bị loại theo điều kiện R^2.

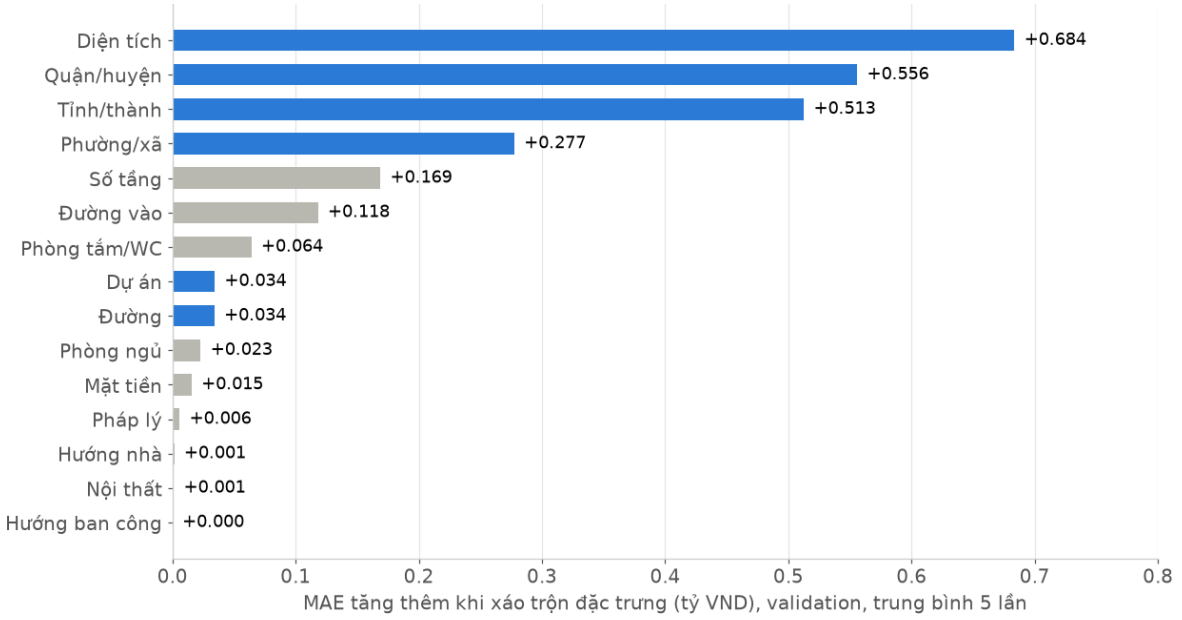
- **Ensemble:** trọng số XGBoost được thử từ 0 đến 1, bước 0.1, trên dự đoán out-of-fold; 0.5 cho MAE thấp nhất, thắng cả 3 fold so với XGBoost đơn lẻ. Trọng số 0.4–0.6 cho kết quả gần như bằng nhau.

Bảng 17: Cấu hình mô hình cuối (số cây chọn bằng dừng sớm trên 15% tập train, chia theo nhóm, rồi refit trên toàn bộ train của `splits_v3`)

Thành phần	Tham số	Tiền xử lý	Số cây
XGBoost (0.5)	`max_depth`=9, `min_child_weight`=1, `subsample`=0.8, `colsample_bytree`=0.7, `learning_rate`=0.05	Trung vị cho mặt tiền/đường vào; One-Hot; target encoding phường, đường, dự án	1 043
LightGBM (0.5)	`num_leaves`=63, `min_child_samples`=5, `colsample_bytree`=0.7, `learning_rate`=0.05	Như trên, không điền mặt tiền/đường vào	683

VII.4 Mô hình học được điều gì?

Nhóm đo *permutation importance* của mô hình cuối (fit trên `splits_v3`) trên tập validation, phần dữ liệu không dùng để chọn gì ở bước cải tiến: xáo trộn giá trị một biến đầu vào, giữ nguyên các biến khác, và đo MAE tăng thêm bao nhiêu (trung bình 5 lần xáo trộn). Kết quả tính theo biến người dùng nhập thay vì hàng trăm cột sau mã hóa. Xáo trộn tỉnh hoặc quận cũng làm hỏng khóa phường/đường ghép từ chúng, nên hai biến này mang cả một phần thông tin vị trí chi tiết.



Hình 8: Permutation importance của mô hình cuối trên tập validation (màu xanh: diện tích và các biến vị trí)

- **Vị trí là nhóm mô hình dựa vào nhiều nhất:** quận (+0.556 tỷ VND), tỉnh (+0.513), phường/xã (+0.278), tên dự án (+0.034), tên đường (+0.034). Kết quả đồng ý với H1, phát hiện 1 và phát hiện 7.
- **Diện tích đứng đầu khi xét riêng lẻ** (+0.684 tỷ VND), dù tương quan đơn biến chỉ 0.19. Điều này phù hợp với cơ chế "giá ≈ đơn giá khu vực × diện tích": diện tích chỉ có nghĩa khi đã biết vị trí, và là một cách giải thích cho nghịch lý ở mục VI.2.

- **Kết cấu và hạ tầng**: số tầng (+0.169), đường vào (+0.118), phòng tắm (+0.064) xếp tiếp theo, khớp với H4 và với thực tế nhà phố nhiều tầng, ngõ ô tô vào được có giá cao hơn.

- **Pháp lý, hướng nhà, nội thất, hướng ban công** đều dưới 0.01 tỷ VND: khi đã biết vị trí và kết cấu, mô hình gần như không dùng thêm các biến này. **Điều đó không có nghĩa chúng không quan trọng với người mua ngoài thực tế.** Permutation importance chỉ đo mô hình này dựa vào biến nào trên dữ liệu này: các biến trên bị bỏ trống nhiều (hướng nhà 70%, nội thất 47%), được ghi thô (nội thất chỉ "đầy đủ" hoặc "cơ bản"), và phần ảnh hưởng của chúng có thể đã nằm trong vị trí hoặc loại nhà. SHAP (TreeSHAP gộp theo biến) cho cùng bức tranh: diện tích đứng đầu, nhóm vị trí và kết cấu theo sau, pháp lý, hướng và nội thất ở cuối; riêng trong nhóm vị trí, SHAP xếp phường/xã cao hơn tỉnh và quận.

Quay về domain

Thứ hạng của mô hình giống cách một môi giới có kinh nghiệm định giá: hỏi "ở đâu" trước, rồi "bao nhiêu mét", rồi "mấy tầng, ngõ rộng không". Các yếu tố được nhắc nhiều trong tin rao như hướng nhà hay nội thất hầu như không được mô hình dùng tới trong dữ liệu này; muốn biết chúng ảnh hưởng giá thật bao nhiêu cần dữ liệu ghi đầy đủ hơn, không thể kết luận từ mô hình này.

VIII Evaluation

VIII.1 Chỉ số kỹ thuật

Mô hình cuối được đánh giá theo ba cách. *Out-of-fold*: mỗi dòng train được dự đoán bởi pipeline của fold không chứa dòng đó. *Validation*: mô hình fit trên toàn bộ train, chấm trên validation. *Test*: chấm sau khi đã khóa cấu hình. Lần đánh giá đầu (20/10/2026) dùng các tập chia ngẫu nhiên theo dòng (`splits_v2`). Vì khoảng 10.7% căn test có tin đăng lại nằm trong train, theo góp ý phản biện nhóm chia lại dữ liệu theo nhóm tin cùng căn (`splits_v3`, mục V.3), fit lại đúng cấu hình đó và đánh giá lại (notebook 05, 26/10/2026). Tập test mới gần như gồm cùng các căn với tập test cũ, nên đây là lần đánh giá lại chứ không phải một tập chưa từng xem.

Bảng 18: Kết quả mô hình cuối (tỷ VND): chia tập ngẫu nhiên so với chia theo nhóm tin cùng căn

Tập đánh giá	**MAE**	**RMSE**	R^2
Chia ngẫu nhiên theo dòng (`splits_v2`, notebook 03)			
3-fold CV, out-of-fold (train)	0.915	1.243	0.685
Validation	0.880	1.210	0.695
Test	0.892	1.219	0.699
Test, bỏ 10.7% dòng có tin trùng trong train	0.934	1.266	0.677
Chia theo nhóm tin cùng căn (`splits_v3`, notebook 05), dùng trong ứng dụng			
3-fold CV theo nhóm, out-of-fold (train)	0.930	1.260	0.676
Validation	0.894	1.224	0.686
Test (đánh giá lại)	**0.921**	**1.248**	**0.686**
Test, chỉ 77.9% căn không có tin cùng địa chỉ và diện tích trong train	0.979	1.318	0.660
Test, XGBoost đơn lẻ	0.927	1.259	0.681
Test, LightGBM đơn lẻ	0.931	1.258	0.681

Khi chia theo nhóm, mọi chỉ số đều kém đi một chút: MAE CV tăng từ 0.915 lên 0.930, R^2 test giảm từ 0.699 xuống 0.686, gần với con số 0.677 khi bỏ tin trùng ở lần đánh giá đầu. Phần chênh này là lợi thế mô hình có được nhờ "nhớ" giá của tin đăng lại; con số mới gần hơn với việc ước tính một căn chưa từng thấy. Phép kiểm tra chặt hơn (chỉ giữ căn test không có tin nào cùng địa chỉ và diện tích trong train, kể cả căn khác cùng dự án) cho $R^2 = 0.660$. Thứ tự giữa các cách đánh giá không đổi: ensemble tốt hơn từng mô hình đơn lẻ, và test sát validation. Khoảng cách train/CV (0.58 so với 0.92 tỷ VND với XGBoost) cho thấy mô hình còn overfit, nhưng dừng sớm đã giữ sai số trên dữ liệu chưa thấy ổn định.

Kết quả nghiệm thu

Mục tiêu $R^2 \geq 0.7$ trên test **chưa đạt**: $R^2 = 0.686$ khi chia theo nhóm tin cùng căn (0.699 ở lần chia ngẫu nhiên, cũng chưa đạt). Cấu hình không được chỉnh lại sau khi xem test; mọi cải tiến sau này phải được chọn bằng CV theo nhóm và báo cáo rõ là tập test đã được xem trước.

VIII.2 Chỉ số theo domain và giá trị thực tế

Người mua không hỏi "MAE bao nhiêu" mà hỏi "con số này lệch tối đa bao nhiêu phần trăm". Bảng dưới dịch kết quả sang các chỉ số đó và so với hai cách làm hiện tại. Hai mốc so sánh học từ phần train của từng fold (khi chấm test: từ toàn bộ train); quy tắc đơn giá dùng trung vị đơn giá của quận, quận chưa có thì lấy của tỉnh. Mọi cột dùng cách chia theo nhóm tin cùng căn (`splits_v3`).

Bảng 19: Kết quả theo chỉ số domain (out-of-fold theo nhóm trên 18 880 căn train; cột cuối: test, 4 274 căn)

Chỉ số domain	Trung vị giá	Đơn giá quận × diện tích	LightGBM	XGBoost	Ensemble	Ensemble (test)
% căn có sai số ≤ 10%	18.9%	28.6%	41.7%	41.4%	**42.0%**	42.1%
% căn có sai số ≤ 20%	36.6%	52.0%	69.2%	69.1%	**69.7%**	70.1%
Sai số tương đối trung vị (MdAPE)	26.9%	19.1%	12.5%	12.6%	**12.5%**	12.2%
% căn lệch quá 1 tỷ	68.0%	51.5%	35.7%	36.0%	**35.5%**	35.0%
MAE (tỷ VND)	1.846	1.511	0.937	0.941	**0.929**	0.921

So với quy tắc đơn giá quận × diện tích, mô hình cuối nâng tỷ lệ căn được ước lượng trong ±20% từ 52% lên 70% (70% trên test, so với 51% của quy tắc) và giảm sai số tương đối trung vị từ 19% xuống 12.5%. Với một căn 6 tỷ VND, đó là chênh lệch giữa sai khoảng 1.1 tỷ VND và khoảng 0.7 tỷ VND ở trường hợp điển hình. Tuy vậy, gần ba căn thì vẫn có một căn lệch hơn 20%, nên con số chỉ nên dùng làm điểm tham chiếu.

VIII.3 Khoảng dự đoán

Một con số đơn lẻ không cho người dùng biết nó có thể lệch bao xa. Notebook `notebooks/04_calibration.ipynb` thêm **khoảng dao động 80%** bằng split conformal prediction. Cách này không giả định phân phối sai số; nó chỉ dùng sai số của mô hình đã khóa trên tập calibration

(chưa tham gia huấn luyện hay chọn mô hình). Phương pháp được chọn ở notebook 04 với mô hình chia ngẫu nhiên; sau khi chia lại theo nhóm tin cùng căn, notebook 05 học lại khoảng bằng đúng phương pháp đó trên tập calibration mới (2 807 dòng) mà không chọn lại. Hai cách được so:

- **khoảng đều** $\hat{y} \pm q$, với q là phân vị 80% (có hiệu chỉnh mẫu hữu hạn) của $|y - \hat{y}|$;
- **khoảng theo nhóm giá** (Mondrian conformal): chia theo 5 nhóm giá dự đoán, mỗi nhóm có hai đầu $[\hat{y} + q_{lo}, \hat{y} + q_{hi}]$ là phân vị 10% và 90% của sai số trong nhóm.

Cả hai được cắt về khoảng giá có trong dữ liệu (1–11.5 tỷ VND) và luôn chứa giá ước tính. Quy tắc chọn được viết trước khi chạy: chọn cách có độ rộng trung bình nhỏ nhất trong các cách có độ phủ trên validation từ 78% trở lên và không nhóm giá nào dưới 70%. Sau khi khóa, độ phủ được đo **một lần** trên test.

Bảng 20: Khoảng dao động 80%: tỷ lệ căn có giá nằm trong khoảng (độ phủ) và độ rộng

Cách, tập	Độ phủ	Theo nhóm giá	Căn mới*	Độ rộng TB
Chọn phương pháp, chia ngẫu nhiên (`splits_v2`, notebook 04)				
Khoảng đều (±1.42 tỷ VND), validation	80.7%	73.4–90.7%	79.2%	2.82 tỷ VND
Theo nhóm giá, validation	80.8%	80.4–81.3%	79.3%	2.81 tỷ VND
Theo nhóm giá, test	80.0%	75.7–82.3%	78.3%	2.81 tỷ VND
Học lại theo nhóm tin cùng căn (`splits_v3`, notebook 05), dùng trong ứng dụng				
Theo nhóm giá, validation	80.6%	79.4–82.2%	77.8%	2.85 tỷ VND
Theo nhóm giá, test (đánh giá lại)	**78.9%**	**75.4–81.6%**	**76.5%**	**2.85 tỷ VND**

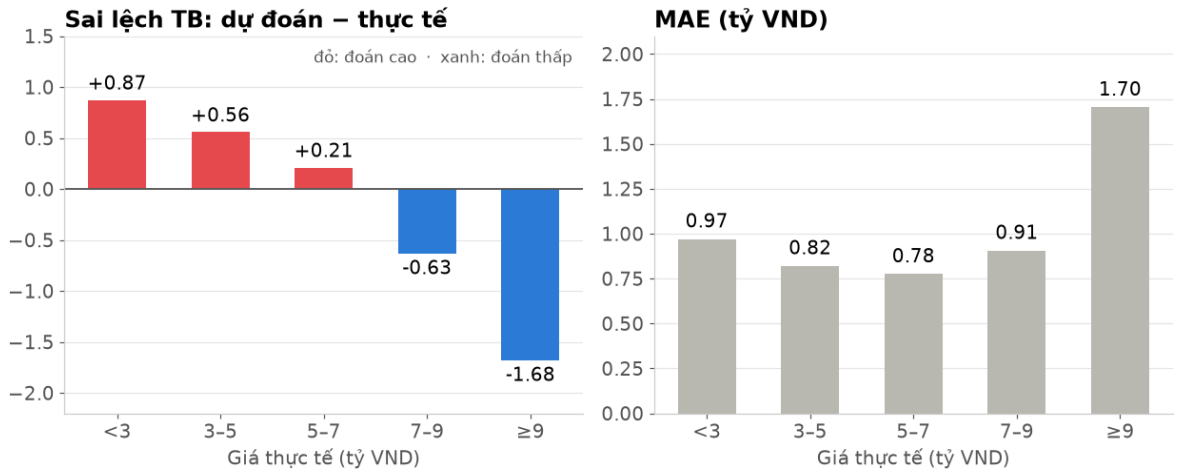
* `splits_v2`: căn không có tin trùng trong train; `splits_v3`: căn không có tin nào cùng địa chỉ và diện tích trong train.

Hai cách có độ phủ tổng thể và độ rộng gần như bằng nhau. Tuy vậy, khoảng đều chỉ phủ 73% ở nhóm giá 6.6–7.5 tỷ VND, nơi giá thật có lúc cao hơn dự đoán rất nhiều, và tới 91% ở nhóm dưới 4.15 tỷ VND, nơi mô hình đoán tốt nhất (MAE 0.62 tỷ VND). Khoảng theo nhóm giá sửa được điều này: nhà rẻ có khoảng hẹp và lệch xuống (−1.15 / +0.70 tỷ VND), còn nhóm 6.6–7.5 tỷ VND có đầu trên rộng hơn (+1.99 tỷ VND). Đây là cách được chọn. Học lại trên tập calibration theo nhóm cho các đầu gần như cũ (nhà rẻ −1.16 / +0.76 tỷ VND; nhóm 6.5–7.5 tỷ VND có đầu trên +1.98 tỷ VND). Ví dụ, căn 60 m^2, 4 tầng ở Phường Dịch Vọng, Cầu Giấy được ước lượng 9.0 tỷ VND, khoảng 7.6–10.4 tỷ VND.

Khi chia theo nhóm, độ phủ trên test là 78.9%, thấp hơn mục tiêu 80% khoảng một điểm, và còn 76.5% với các căn không có tin nào cùng địa chỉ và diện tích trong train. Như vậy với một tin hoàn toàn mới, khoảng "80%" thực tế phủ khoảng 3/4 số căn; ứng dụng hiển thị đúng con số đo được thay vì ghi 80%. Độ phủ là trung bình trên nhiều căn (kể cả trong từng nhóm giá), không phải bảo đảm cho từng căn. Kết quả lưu ở `experiments/group_split/prediction_interval.json`, kèm mã băm của mô hình mà nó được tính cho.

Khoảng rộng 2.85 tỷ VND có giúp quyết định không? Độ rộng trung bình bằng gần một nửa giá trung vị (5.9 tỷ VND): đây là mức bất định thật của mô hình khi chỉ biết các thông tin trong form. Nhận định của nhóm, *chưa được kiểm chứng với người dùng*, là khoảng này giúp được một việc hẹp: nhận ra một mức giá rao nằm xa hẳn ngoài khoảng (ví dụ cao hơn đầu trên hơn 1 tỷ VND), để hỏi thêm hoặc so sánh thêm; nó không đủ hẹp để chốt giá mua bán. Ứng dụng nói rõ điều đó ngay dưới khoảng. Việc người dùng có hiểu và thấy khoảng này hữu ích hay không được đưa vào kế hoạch kiểm thử ở mục IX.2.

VIII.4 Error Analysis



Hình 9: Sai lệch trung bình (trái) và MAE (phải) của mô hình cuối theo nhóm giá thực tế (out-of-fold theo nhóm tin cùng căn)

Bảng 21: Sai số mô hình cuối theo nhóm điều kiện (out-of-fold theo nhóm tin cùng căn, train của `splits_v3`)

Nhóm	n	MAE	Sai lệch TB	MdAPE
Quận có ≥ 100 tin train	15 974	0.895	−0.03	11.8%
Quận có 30–99 tin	1 908	0.983	+0.03	15.7%
Quận có < 30 tin	995	1.378	+0.25	24.0%
Thiếu 0–2 trường tùy chọn	6 931	0.855	−0.02	12.0%
Thiếu 3–5 trường	8 497	0.924	−0.01	12.3%
Thiếu 6–9 trường	3 452	1.092	+0.03	14.2%
Diện tích < 50 m^2	7 425	0.820	+0.02	12.2%
Diện tích ≥ 200 m^2	434	1.546	−0.06	21.3%

Bốn nhóm lỗi chính và nguyên nhân theo domain:

1. **Kéo về trung bình ở hai đầu giá.** Căn dưới 3 tỷ VND bị đoán cao trung bình 0.87 tỷ VND, căn từ 9 tỷ VND bị đoán thấp 1.68 tỷ VND; trên test lần lượt +0.81 và −1.71 tỷ VND. Hai căn cùng phường, cùng diện tích có thể chênh giá vì mặt đường hay ngõ sâu, nhà mới hay nhà cũ, những thông tin không có trong dữ liệu; mô hình vì thế dự đoán gần mức trung bình của nhóm. Dự đoán chỉ trải từ 0.37 đến 10.79 tỷ VND, hẹp hơn khoảng giá thật.

2. **Quận ít dữ liệu.** 995 căn ở các quận có dưới 30 tin train có MAE 1.38 tỷ VND và bị đoán cao (+0.25 tỷ VND): mô hình mượn mức giá của các quận đông dữ liệu, vốn đắt hơn.

3. **Tin khai báo thiếu và nhà rất rộng.** Thiếu 6–9 trường làm MAE tăng 28% so với tin khai báo gần đủ; nhà từ 200 m^2 có MAE 1.55 tỷ VND vì ít mẫu và đơn giá biến động mạnh ở vùng ven.

4. **Dữ liệu nghi sai.** Đọc 20 mẫu sai nhiều nhất (notebook 03, mô hình chia ngẫu nhiên) thấy hai tin cùng dự án, cùng diện tích, số tầng và số phòng nhưng giá 3.2 và 9.5 tỷ VND, giá rất thấp ở quận trung tâm (Quận 1, 3, Bình Thạnh: 2.3–3.1 tỷ VND cho nhà 3–4 tầng) và một lỗi diện tích gần như chắc chắn (4 m^2). Riêng nhóm tin có đơn giá dưới 1/3 mặt bằng quận (130 tin) đã chiếm 15% trong 200 lỗi lớn nhất; phần lớn là đất rộng ở xã ngoại thành, rẻ thật chứ không hẳn sai. Nhóm không xóa các dòng này vì chưa xác minh được với nguồn.

IX Application & Recommendations

IX.1 Cách sử dụng kết quả

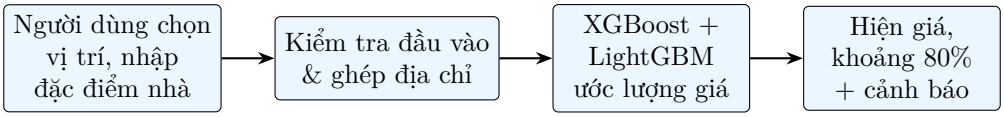
Mô hình cuối đã được khóa và xuất thành file (`models/candidates/final_v3_group_2026-10-26/house_price_ensemble.pkl`, kèm manifest; fit trên train của `splits_v3`); nạp lại trong một tiến trình Python mới cho dự đoán trùng khớp. Nhóm xây dựng hai bản ứng dụng cho người dùng phổ thông, toàn bộ tiếng Việt, dùng chung trường nhập, quy tắc kiểm tra, nội dung chữ và số liệu:

- **Bản Streamlit** (`app/streamlit_app.py`) gọi mô hình qua một điểm duy nhất (`app/core/model_interface.py`). File `.pkl` chỉ được nạp khi mã SHA-256 khớp manifest và máy chủ chạy đúng Python 3.14 cùng các phiên bản thư viện lúc lưu (cloudpickle lưu mã của notebook dưới dạng bytecode Python 3.14); nếu không, ứng dụng khóa nút ước tính và ghi rõ lý do. Bản này deploy trên Streamlit Community Cloud (chọn Python 3.14) hoặc bằng `Dockerfile`.

- **Bản web tĩnh** (`web/public/`), deploy trên Vercel. Vercel không chạy được máy chủ Streamlit, nên script `web/scripts/export_web_data.py` xuất toàn bộ tham số tiền xử lý của mô hình đã khóa (chuẩn hóa, điền khuyết, one-hot, target encoding) cùng 1 043 cây XGBoost và 683 cây LightGBM ra JSON, và `web/public/model.js` tính dự đoán ngay trong trình duyệt; thông tin người dùng nhập không được gửi đi. Kiểm thử đối chiếu với file `.pkl` trên toàn bộ 4 268 dòng validation và 2 807 dòng calibration cho chênh lệch bằng 0; khi tải trang, ứng dụng tự tính lại ba căn mẫu và dừng nếu kết quả lệch.

Cả hai bản đã được kiểm tra cho ra cùng con số với việc gọi mô hình trực tiếp (ví dụ Hà Nội, Cầu Giấy, Phường Dịch Vọng, 60 m^2, 4 tầng: 9.0 tỷ VND). Mọi trang của ứng dụng đều ghi rõ dữ liệu là **tin rao bán năm 2024**: ở lời giới thiệu, ngay dưới con số ước tính ("giá hiện nay có thể đã khác"), ở phần so sánh với dữ liệu và trong danh sách giới hạn.

Mô hình cần **địa chỉ chi tiết**: khi không có phường/xã, tên đường và dự án, MAE trên validation (notebook 03, `splits_v2`) tăng từ 0.88 lên 1.14 tỷ VND và R^2 giảm từ 0.695 còn 0.556, thấp hơn nhiều so với R^2 khoảng 0.64 mà cấu hình không dùng địa chỉ chi tiết đạt trong CV (hai con số đo trên hai tập khác nhau, chỉ để tham khảo). Vì vậy giao diện bắt buộc chọn phường/xã từ danh sách có trong dữ liệu của quận đó; tên đường và dự án là tùy chọn.



Hình 10: Quy trình ước lượng giá trong ứng dụng

Người dùng chọn vị trí từ danh sách (tỉnh, quận, phường/xã; tên đường và dự án nếu có) và nhập đặc điểm căn nhà. Chỉ bốn mục bắt buộc: tỉnh, quận, phường/xã và diện tích; các mục còn lại để "Không rõ" được, vì mô hình đã học cách xử lý giá trị thiếu. Danh sách chỉ gồm 108 cặp tỉnh–quận có ít nhất 10 tin train, cùng các phường, đường, dự án đã xuất hiện trong dữ

liệu của quận đó. Kết quả hiển thị là *ước lượng giá rao* bằng tỷ VND (làm tròn 0.1 tỷ VND, kèm cách đọc bằng chữ), không gọi là giá giao dịch hay giá thẩm định. Đi kèm là **khoảng dao động 80%** (mục khoảng dự đoán ở phần Evaluation; ví dụ 7.6–10.4 tỷ VND cho căn ở Cầu Giấy ở trên) cùng câu giải thích độ phủ đo được trên tập test (79%) và cách dùng khoảng (để nhận ra mức giá quá xa, không để chốt giá), sai số trung bình 0.92 tỷ VND đo trên tập test, nguồn mô hình, và vị trí của mức giá cùng khoảng của nó so với 18 880 căn năm 2024 trong tập train. Cả hai bản ứng dụng chỉ dùng file khoảng khi mã băm ghi trong đó khớp với mô hình đang chạy; bản web còn được kiểm thử cho ra đúng khoảng như bản Python trên mọi dòng validation và calibration. Cảnh báo xuất hiện khi một giá trị nhập nằm ngoài khoảng có trong dữ liệu, khi phường/xã có dưới 5 tin train, khi giá ước lượng nằm ngoài 1–11.5 tỷ VND, và khi thông tin đã đổi sau lần ước tính.

IX.2 Kiểm thử với người dùng

Các con số ở mục VIII cho biết mô hình sai bao nhiêu, nhưng không cho biết người dùng có dùng được ứng dụng hay không. Nhóm đã soạn kịch bản kiểm thử (`reports/user_testing.md`) với 6–8 người chưa từng thấy ứng dụng (người đang tìm mua nhà, người bán và môi giới), mỗi buổi khoảng 20 phút, người điều phối không giải thích giao diện. Kịch bản trả lời bốn câu hỏi:

1. **Nhập được thông tin không?** Người dùng tự nhập một căn họ biết; đo số người hoàn thành không cần trợ giúp, thời gian, các mục bị bỏ trống hoặc hiểu sai (nhất là phường/xã, mặt tiền, đường vào).
2. **Hiểu khoảng giá không?** Hỏi người dùng giải thích lại "khoảng 7.6–10.4 tỷ VND" bằng lời của họ và trả lời hai câu kiểm tra: giá thật có chắc nằm trong khoảng không; nếu giá rao 12 tỷ VND thì nghĩ gì.
3. **Khoảng rộng khoảng 2.8 tỷ VND có giúp quyết định không?** Cho ba tin rao (một tin trong khoảng, một tin cao hơn đầu trên, một tin thấp hơn đầu dưới) và hỏi người dùng sẽ làm gì; chấm thang 1–5 mức hữu ích.
4. **Có nhận ra dữ liệu là năm 2024 không?** Hỏi sau khi dùng, không gợi ý trước.

Tiêu chí đạt được đặt trước: ít nhất 5/6 người tự nhập xong, ít nhất 4/6 người giải thích đúng khoảng, và ít nhất 4/6 người nhận ra dữ liệu năm 2024.

Trạng thái

Tại thời điểm nộp báo cáo, buổi kiểm thử **chưa được thực hiện**; báo cáo không đưa ra kết quả nào về người dùng. Các nhận định về tính hữu ích của khoảng ở mục VIII.3 là ý kiến của nhóm. Kết quả sẽ được ghi vào `reports/user_testing.md` và có thể dẫn tới sửa câu chữ hoặc cách hiển thị khoảng.

IX.3 Khuyến nghị

Bảng 22: Khuyến nghị dựa trên data

Khuyến nghị	Căn cứ từ data	Người thực hiện	Tác động	Rủi ro
Bắt buộc nhập tỉnh, quận và phường/xã; khuyến khích nhập tên đường, dự án; gợi ý tên từ danh sách chuẩn hóa	Tỉnh, quận: importance ≈ 0.48 tỷ VND mỗi biến; phường giảm MAE 4.5%; thiếu địa chỉ chi tiết: R^2 0.56	Nhóm app	Giảm lỗi lớn nhất	Người dùng không biết phường; tên đường viết nhiều kiểu
Giữ phường/xã là trường bắt buộc (đã làm trong app); nếu muốn cho bỏ trống, huấn luyện thêm mô hình không dùng địa chỉ chi tiết	Thiếu phường/đường/dự án: R^2 validation chỉ còn 0.556	Nhóm model, nhóm app	Tránh ước lượng sai lệch lớn	Người dùng không biết phường; phải bảo trì hai mô hình
Cảnh báo khi vị trí ít dữ liệu hoặc giá dự đoán ngoài 1–11.5 tỷ VND (app đã cảnh báo phường < 5 tin và giá ngoài khoảng; còn thiếu cảnh báo quận < 30 tin)	MAE 1.38 ở quận ít tin	Nhóm app	Tránh tin nhầm con số yếu	Người dùng bỏ qua cảnh báo
Hiện khoảng giá cùng con số (đã làm: khoảng 80% theo nhóm giá, độ phủ test 79%, căn mới 76.5%); tiếp theo: khoảng hẹp hơn, thích ứng theo từng căn; kiểm thử với người dùng	Sai lệch +0.87 / −1.68 tỷ VND ở hai đầu; khoảng rộng trung bình 2.85 tỷ VND	Nhóm model, nhóm app	Phản ánh đúng độ bất định	Khoảng quá rộng để chốt giá; người dùng hiểu nhầm là chắc chắn
Xác minh 238 tin bất thường; giữ cách chia theo nhóm tin cùng căn (đã làm) cho mọi lần đánh giá sau	Lỗi đơn vị (4 m^2); tin trùng làm R^2 lạc quan 0.013	Nhóm dữ liệu	Điểm số trung thực hơn	Ghép nhóm sót tin cùng căn ghi địa chỉ khác
Người mua: so giá rao với khoảng ước lượng, đặc biệt nhà diện tích lớn	Nghịch lý diện tích; MdAPE 12%	Người dùng	Có cơ sở thương lượng	Thiếu thông tin chất lượng nhà
Người bán, môi giới: khai báo đủ số tầng, WC, đường vào	Thiếu 6–9 trường: MAE tăng 28%	Người bán	Ước lượng sát hơn	Khai báo sai

IX.4 Giám sát và cập nhật

- **Trước khi phát hành:** cấu hình đã khóa, đánh giá lại trên test chia theo nhóm tin cùng căn ($R^2 = 0.686$, notebook 05); khoảng dự đoán 80% đã hiệu chỉnh trên calibration và đo độ phủ theo nhóm giá. Còn lại: kiểm thử với người dùng (mục IX.2). Cũng cần chuyển mã tiền xử lý từ notebook ra một module Python để lưu mô hình bằng pickle/joblib thường thay cho cloudpickle. Ứng dụng đã nạp mô hình, có ô nhập phường/xã, đường, dự án và hiện khoảng dao động. Mỗi khi đổi mô hình phải học lại khoảng (notebook 05; khoảng gắn với mã băm của mô hình), `web/scripts/export_web_data.py` và bộ kiểm thử để cả hai bản ứng dụng dùng đúng mô hình mới. Nếu refit trên train + validation thì không chấm lại test.

- **Độ phủ của khoảng:** khi có giá thật của các căn đã ước lượng, đo lại tỷ lệ giá nằm

trong khoảng theo từng nhóm giá; nếu tụt rõ dưới 80% thì hiệu chỉnh lại khoảng trên dữ liệu mới.

- **Drift dữ liệu đầu vào:** theo dõi phân bố tỉnh/quận, diện tích và tỷ lệ bỏ trống của người dùng so với tập train; tỷ lệ quận "chưa từng thấy" tăng là tín hiệu cần bổ sung dữ liệu.

- **Drift giá:** dữ liệu là ảnh chụp năm 2024; khi có tin rao mới, so trung vị đơn giá theo quận với dữ liệu cũ. Nếu lệch hơn 10% ở các quận lớn, huấn luyện lại thay vì tiếp tục dùng mô hình cũ.

X Limitations & Future Work

- **Giới hạn dữ liệu:** giá rao, không phải giá giao dịch; chỉ khoảng 1–11.5 tỷ VND, nên mô hình không có cơ sở cho nhà rẻ hơn hoặc đắt hơn; 74% tin ở Hà Nội và TP.HCM; một thời điểm năm 2024. Đơn vị của diện tích, mặt tiền, đường vào chưa được nguồn mô tả rõ.

- **Thông tin thiếu:** không có tọa độ, năm xây dựng, loại hình (nhà dự án/thổ cư, mặt phố/trong ngõ); phường và tên đường lấy được từ địa chỉ nhưng nhiều đường chỉ có vài tin. Đây là nguyên nhân chính của hiện tượng kéo về trung bình.

- **Chưa đạt nghiệm thu:** R^2 test 0.686 khi chia theo nhóm tin cùng căn (0.699 khi chia ngẫu nhiên), dưới mục tiêu 0.7; với căn không có tin cùng địa chỉ và diện tích trong train chỉ 0.660. Khoảng "80%" rộng trung bình 2.85 tỷ VND, độ phủ test 78.9% và 76.5% với căn mới; chỉ là độ phủ trung bình (theo nhóm giá), không cho từng căn. Chưa kiểm thử với người dùng và chưa phỏng vấn chuyên gia domain.

- **Kết luận về biến:** các kết luận như "pháp lý, hướng nhà không giúp dự đoán" chỉ nói về dữ liệu này và mô hình này. Biến được mô hình dùng ít có thể vẫn quan trọng ngoài thực tế (ví dụ vì gần như mọi tin đều ghi có sổ, hoặc vì thông tin đã nằm sẵn trong vị trí).

- **Tin đăng trùng và dữ liệu nghi sai:** khoảng 11% tin trùng địa chỉ, diện tích, giá. Cách chia theo nhóm chỉ ghép được tin cùng địa chỉ và diện tích; tin cùng căn nhưng ghi địa chỉ khác vẫn có thể nằm ở hai tập. 238 tin bất thường chưa được xác minh với nguồn.

- **Phụ thuộc địa chỉ chi tiết:** mô hình cuối kém đi rõ rệt khi không có phường/đường, nên ứng dụng bắt buộc phường/xã và chỉ nhận phường có trong dữ liệu; phường mới hoặc phường chỉ có vài tin được ước lượng kém tin cậy hơn.

- **Giả định chưa kiểm chứng:** mối quan hệ giữa các biến giữ nguyên khi thị trường thay đổi sau 2024.

Hướng phát triển (chọn bằng CV; tập test đã được xem nên phải ghi rõ khi báo cáo):

1. Tọa độ của phường/đường (geocoding): cho mô hình biết khoảng cách và vùng lân cận, giúp cả các phường ít tin; cần đồng ý gửi địa chỉ sang dịch vụ bên ngoài.

2. Target encoding theo tầng (đường → phường → quận → tỉnh) để khóa hiếm co về mức giá của khu vực lớn hơn thay vì trung bình cả nước; thêm CatBoost vào ensemble; chuẩn hóa tên địa phương còn sót ("TP. HCM", "quận Tân Bình").

3. Ghép nhóm tin cùng căn chặt hơn (so khớp tên đường, dự án gần đúng) và chọn lại tham số bằng CV theo nhóm (đã chia theo nhóm và đánh giá lại, nhưng tham số vẫn là tham số chọn trên `splits_v2`); xác minh và sửa các lỗi dữ liệu đã phát hiện.

4. Khoảng dự đoán hẹp hơn và thích ứng theo từng căn (conformalized quantile regression) thay cho khoảng theo nhóm giá hiện tại; SHAP cho từng ước lượng để giải thích trong ứng dụng.

5. Thu thập dữ liệu mới có ngày đăng để đo biến động giá theo thời gian; kiểm chứng với môi giới; thực hiện kiểm thử với người dùng theo `reports/user_testing.md`.

A Data Dictionary đầy đủ

Bảng 23: Toàn bộ biến trong dữ liệu sau khi tách địa chỉ (% thiếu trên 30 229 dòng gốc)

Biến	Kiểu	Đơn vị	Ý nghĩa, giá trị	% thiếu
`Price`	Số thực	tỷ VND	Tổng giá rao; 1.0–11.5	0.0
`Area`	Số thực	m^2	Diện tích ghi trên tin; 3.1–595	0.0
`Frontage`	Số thực	m	Mặt tiền; 1–77 (giá trị gốc)	38.3
`Access Road`	Số thực	m	Độ rộng đường vào; 1–85	44.0
`Floors`	Số nguyên	tầng	1–10	11.9
`Bedrooms`	Số nguyên	phòng	1–9	17.1
`Bathrooms`	Số nguyên	phòng	1–9	23.4
`House direction`	Phân loại	–	8 hướng (Đông, Tây - Nam, ...)	70.3
`Balcony direction`	Phân loại	–	8 hướng	82.6
`Legal status`	Phân loại	–	`Have certificate`, `Sale contract`	14.9
`Furniture state`	Phân loại	–	`Full`, `Basic`	46.7
`City`	Phân loại	–	Phần cuối của `Address`; 106 chuỗi gốc, 76 sau chuẩn hóa	0.0
`District`	Phân loại	–	Phần áp chót của `Address`; 341 chuỗi gốc	0.0
`Ward`	Phân loại	–	Phường/xã/thị trấn, phần đứng trước quận trong `Address`	0.7
`Street`	Phân loại	–	Tên đường/phố/quốc lộ, phần đứng trước phường	14.3
`Project`	Phân loại	–	Tên dự án khi địa chỉ bắt đầu bằng "Dự án"	87.9
`row_id`	Số nguyên	–	Khóa truy vết dòng; không phải đặc trưng	0.0

B Reproducibility & Code Structure

- **Seed:** 42 cho chia tập, fold CV, chia trong fold để early stopping và mô hình.

- **Phiên bản thư viện:** Python 3.14; numpy 2.5.3; pandas 3.0.6; scikit-learn 1.9.1; scipy 1.18.1; xgboost 3.4.1; lightgbm 4.7.0; catboost 1.2.10; joblib 1.6.0; cloudpickle 3.1.2 (môi trường `.venv` của repo). XGBoost lấy mẫu hàng/cột nên kết quả thay đổi nhẹ theo số luồng CPU; LightGBM và CatBoost tái lập chính xác.

- **Notebook:** `notebooks/03_model_experiments.ipynb`, Phần B (C52–C68) chứa toàn bộ bước cải tiến; các experiment lưu theo từng fold trong `experiments/v2/`, chạy lại sẽ đọc kết quả đã có. `notebooks/04_calibration.ipynb` (C01–C07, chạy vài giây) học khoảng dự đoán trên calibration và lưu `experiments/calibration/prediction_interval.json` (cho mô hình chia ngẫu nhiên). `notebooks/05_group_split.ipynb` (C01–C10) tạo `splits_v3`, chạy 3-fold CV theo nhóm, fit mô hình cuối, học lại khoảng, chấm test, tính importance và xuất mô hình; kết quả lưu ở `experiments/group_split/`.

- **Dữ liệu:** `splits_v2` được tạo lại từ `splits_v1` bằng script `models/candidates/milestone_v2_r2_2026-10-16/src/preprocessing/repair_input_v2.py` (tham số `--project-root` là thư mục repo); tập train có SHA-256 bắt đầu bằng `bcbc0634`, đã đối chiếu khớp với manifest của checkpoint. `splits_v3` được tạo từ `splits_v2` trong notebook 05; `split_manifest.csv` ghi nhóm và tập của từng `row_id`.
- **Checkpoint cũ:** `models/candidates/milestone_v2_r2_2026-10-16/` chứa 3 pipeline CV của mỗi mô hình (không phải mô hình final), kèm manifest, chỉ số từng fold và mã tiền xử lý.
- **Mô hình cuối:** `models/candidates/final_v3_group_2026-10-26/` (dùng trong ứng dụng) chứa `house_price_ensemble.pkl` (fit trên train của `splits_v3`), `manifest.json` (tham số, chỉ số, mã băm), `README.md` và `requirements-model.txt`. Mô hình của lần đánh giá đầu vẫn giữ ở `models/candidates/final_v2_2026-10-20/`, cấu hình chọn và kết quả test của nó ở `experiments/final_selection/`.
- **Ứng dụng:** `app/` là bản Streamlit; `app/requirements.txt` ghim đủ phiên bản thư viện (Python 3.14), `packages.txt` và `Dockerfile` ở gốc repo dùng khi deploy. `web/` là bản web tĩnh, deploy trên Vercel với Root Directory là `web` (`web/vercel.json`: không có bước build, phục vụ `web/public/`); dữ liệu của trang (`model.json`, `reference.json`) được tạo lại bằng `web/scripts/export_web_data.py`.
- **Kiểm thử:** cài `requirements-dev.txt` (thư viện của app cộng thư viện cho notebook và test), rồi chạy `python -m unittest discover -s tests`: `tests/test_app.py` (10 test chạy ứng dụng Streamlit không cần trình duyệt, gồm khoảng dự đoán) và `tests/test_web_model.py` (6 test đối chiếu giá và khoảng của bản web với file `.pkl`, cần Node.js 22 trở lên).

Cấu trúc thư mục repo

```
AIQS-Real-Estate-Price-Prediction/
|-- data/
|   |-- vietnam_housing_dataset.csv    # du lieu goc, khong sua
|   |-- _v1.csv                         # sau khi tach dia chi
|   `-- processed/
|       |-- splits_v1/                  # train/validation/calibration/test
|       |-- splits_v2/                  # nhu v1, Frontage/Access Road goc
|       `-- splits_v3/                  # chia theo nhom tin cung can
|-- notebooks/
|   |-- cleandata_estate_aiqs.ipynb     # lam sach, tach Address
|   |-- 01_data_handoff_and_split.ipynb
|   |-- 02_baseline_models.ipynb        # Dummy, Linear, Random Forest
|   |-- 03_model_experiments.ipynb      # boosting, CV, cai tien, test
|   |-- 04_calibration.ipynb            # khoang du doan 80% (conformal)
|   |-- 05_group_split.ipynb            # chia theo nhom, danh gia lai
|   |-- boxplot_price_aiqs.ipynb        # bieu do phan bo gia (EDA)
|   `-- evaluatePrize.ipynb             # kiem tra cot Price (EDA)
|-- experiments/                        # ket qua CV, grid, v2,
|                                       #   final_selection, calibration,
|                                       #   group_split
|-- models/candidates/
|   |-- milestone_v2_r2_2026-10-16/     # checkpoint CV cu
|   |-- final_v2_2026-10-20/            # mo hinh chia ngau nhien
|   `-- final_v3_group_2026-10-26/      # mo hinh cuoi (.pkl) + manifest
|-- app/                                # ban Streamlit
|   |-- streamlit_app.py, styles.css    # giao dien
|   |-- requirements.txt                # thu vien ghim (Python 3.14)
```

```
|   `-- core/                          # schema, validation, model_interface
|-- web/                               # ban web tinh (Vercel)
|   |-- public/                        # index.html, app.js, styles.css,
|   |   |                              #   model.js (du doan trong trinh
    duyet)
|   |   `-- data/                      # model.json, reference.json
|   |-- scripts/                       # export_web_data.py, predict.mjs
|   `-- vercel.json                    # Vercel (Root Directory = web)
|-- tests/                             # test_app.py, test_web_model.py
|-- reports/
|   |-- application_report/            # bao cao nay: main.tex, def.tex,
|   |                                  #   title.tex, IMG/ (logo + hinh)
|   `-- user_testing.md                # kich ban kiem thu nguoi dung
|-- vercel.json                        # Vercel khi Root Directory la goc
    repo
|-- Dockerfile, packages.txt           # deploy Docker / Streamlit Cloud
|-- requirements-dev.txt               # thu vien cho notebook va test
`-- README.md
```